

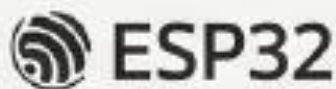
Robótica Industrial

La robótica combina la ciencia y la tecnología para crear máquinas inteligentes capaces de interactuar con el mundo.



El Cerebro del Robot

Un Viaje del Reflejo a la Razón con Arduino, ESP32 y Raspberry Pi



El Espectro del Control



REFLEJO

Control simple, determinista
y en tiempo real.



COMUNICADOR

Inteligencia conectada
para el IoT.



RAZONADOR

Procesamiento complejo
con un sistema operativo.



El Fundamento: Microcontrolador vs. Microprocesador

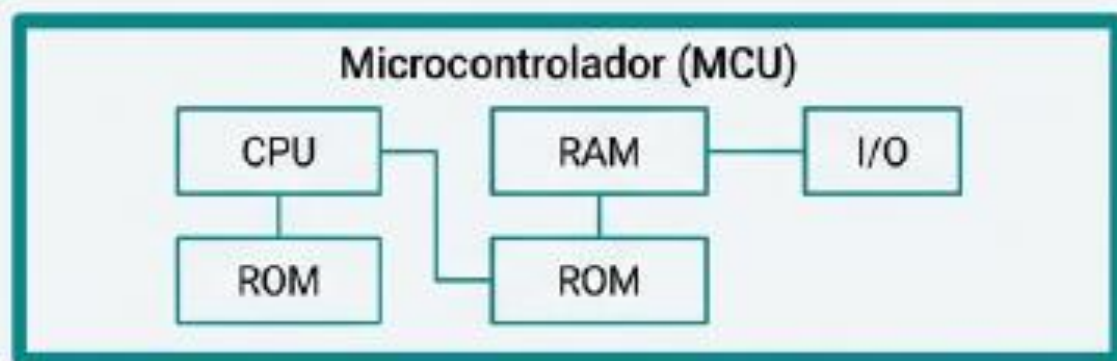
Microcontrolador (MCU)

“El “Cerebro Reptiliano” - un especialista.”

Un chip autocontenido que incluye procesador, memoria (RAM, Flash) y periféricos de entrada/salida (I/O).

Diseñado para una tarea específica, de forma repetitiva y eficiente (determinista). Es el corazón de un dispositivo.

🌀 Arduino, 📶 ESP32



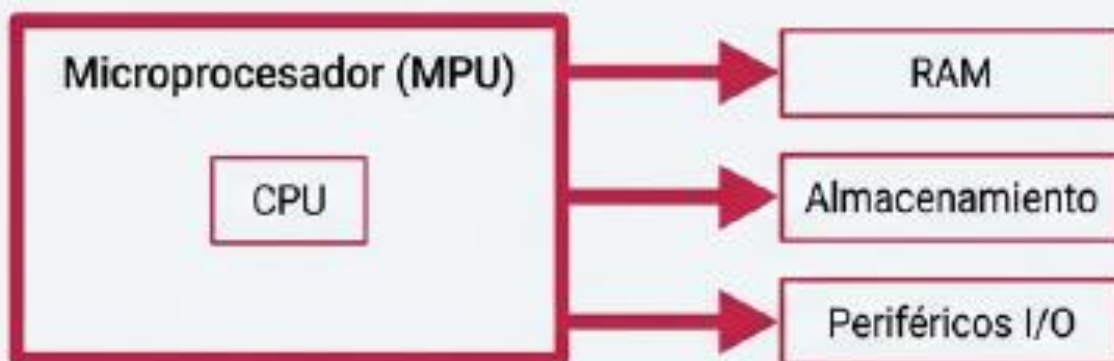
Microprocesador (MPU)

“El “Neocórtex” - un generalista.”

Solo la unidad central de procesamiento (CPU). Necesita componentes externos como RAM, almacenamiento y puertos I/O para funcionar.

Diseñado para ejecutar un sistema operativo completo y manejar múltiples tareas complejas.

🍷 Raspberry Pi, 📱 🖥️, tu laptop, tu smartphone.



Anatomía del Arduino

Identidad:

El maestro del control en tiempo real.

Su propósito es ejecutar un único programa en un bucle infinito con una temporización precisa.

El Corazón (MCU): Microcontrolador Atmel

ATmega328P. Una solución 'todo en uno': CPU, RAM, almacenamiento y periféricos en un solo chip.

Superpoderes:

Ecosistema: Miles de librerías y "shields" (módulos de expansión).

Simplicidad: Abstracción del hardware.

Determinismo: Ejecución predecible y consistente.

El Lenguaje: C++ sobre el framework de Arduino (Wiring). IDE: Arduino IDE, PlatformIO.

La Familia



UNO: El estándar universal



NANO: Potencia en un formato compacto



MEGA: Para proyectos que exigen más pines de E/S.

```
// El clásico "Hola Mundo" del hardware
void setup() {
  // Inicializa el pin digital LED_BUILTIN como salida.
  pinMode(LED_BUILTIN, OUTPUT);
}

// La función loop se ejecuta una y otra vez para siempre
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // Enciende el LED
  delay(1000);                      // Espera un segundo
  digitalWrite(LED_BUILTIN, LOW);  // Apaga el LED
  delay(1000);                      // Espera un segundo
}
```


Anatomía del Arduino UNO

El Corazón

Microcontrolador: Atmel ATmega328P
Arquitectura: 8-bit



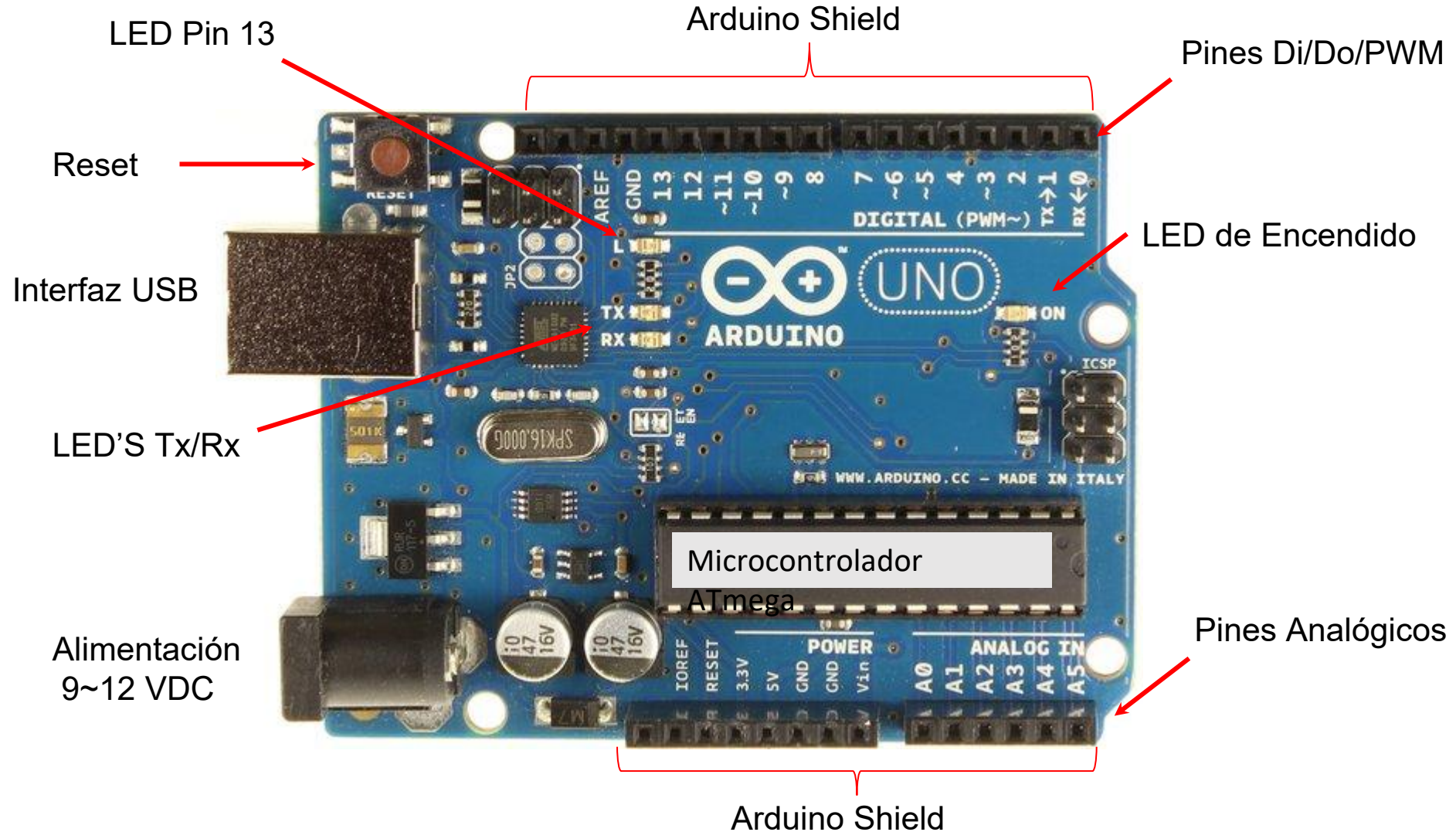
Especificaciones Vitales

Velocidad de Reloj: 16 MHz
Memoria Flash: 32 KB (para tu código)
SRAM: 2 KB (para tus variables)
EEPROM: 1 KB (para datos no volátiles)
Voltaje de Operación: 5V

Arsenal de I/O (Entrada/Salida)

Pines Digitales: 14 (6 PWM)
Pines Analógicos de Entrada: 6
Comunicación: UART, I2C, SPI

Arduino - Hardware

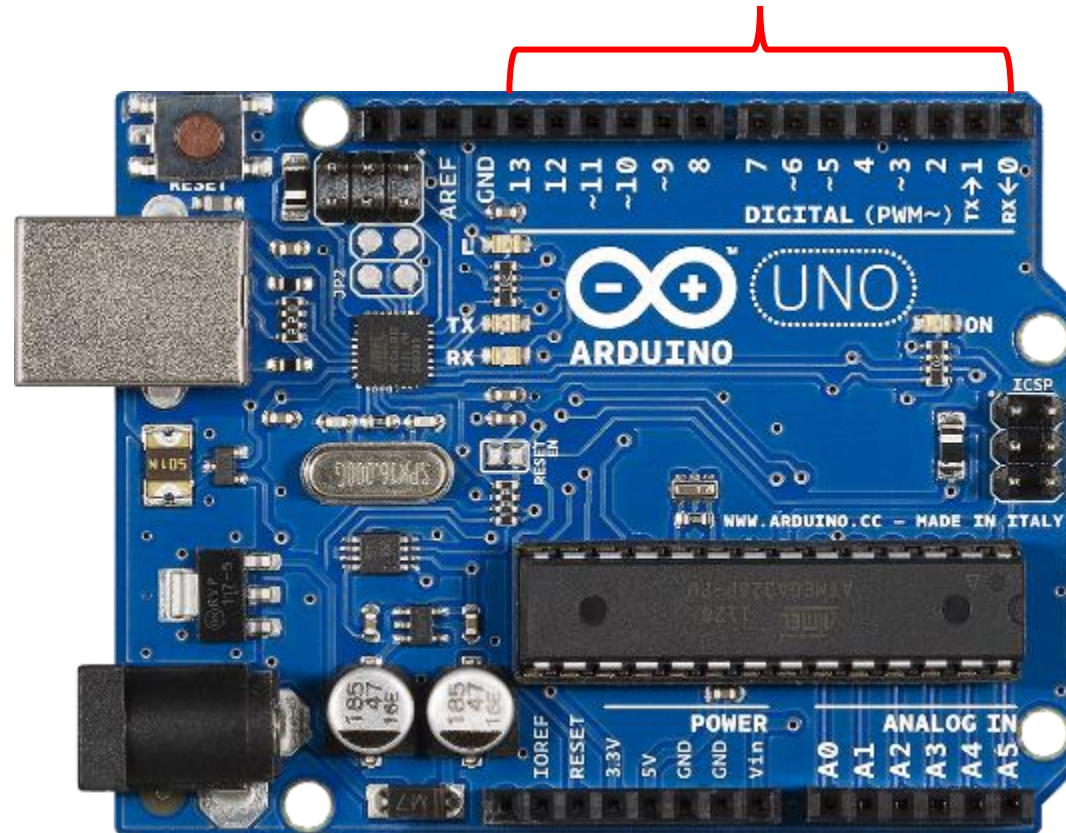


Arduino - Hardware

Señales

Digitales

14 E/S Digitales (Pines 0 al 13)

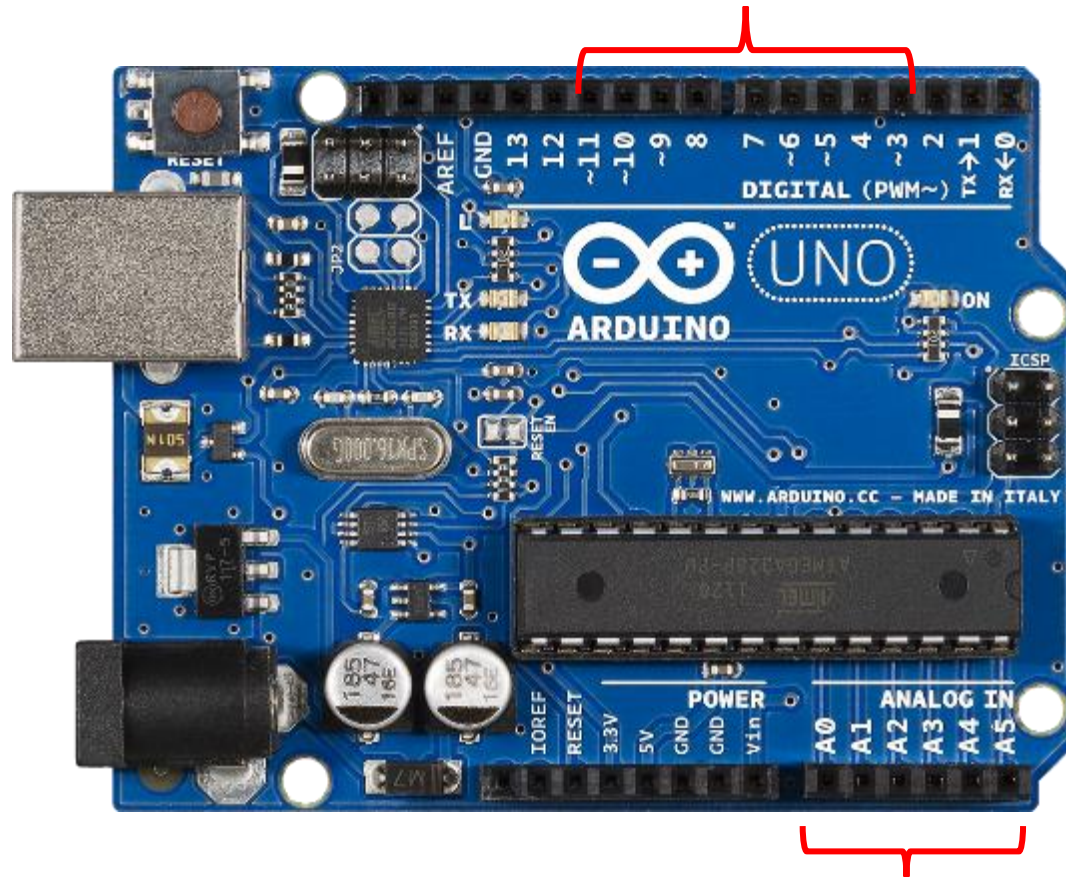


Arduino - Hardware

Señales

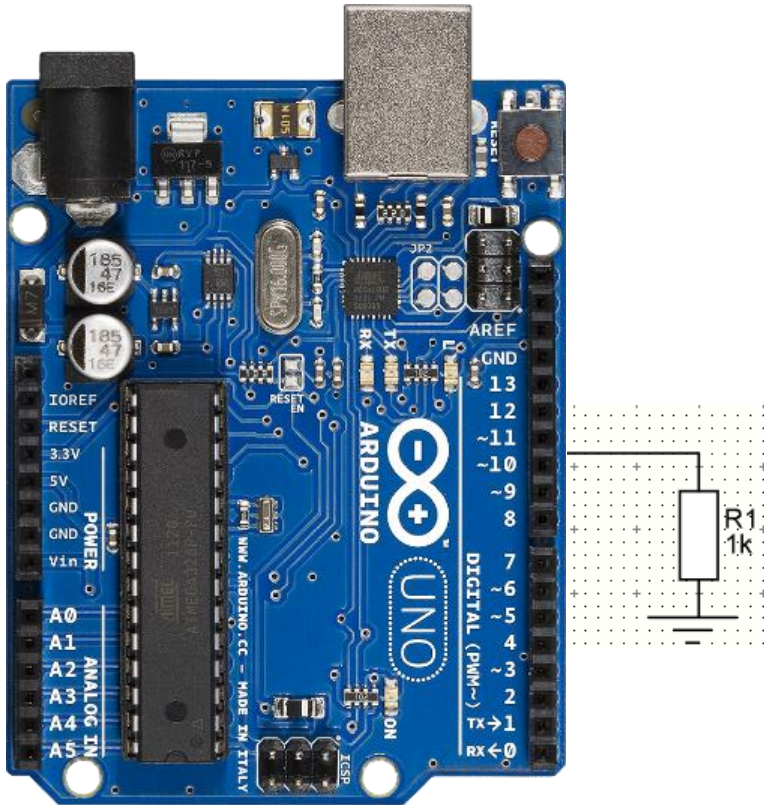
Analogicas

6 Salidas Analógicas (PWM) (Pines 3,5,6,9,11)

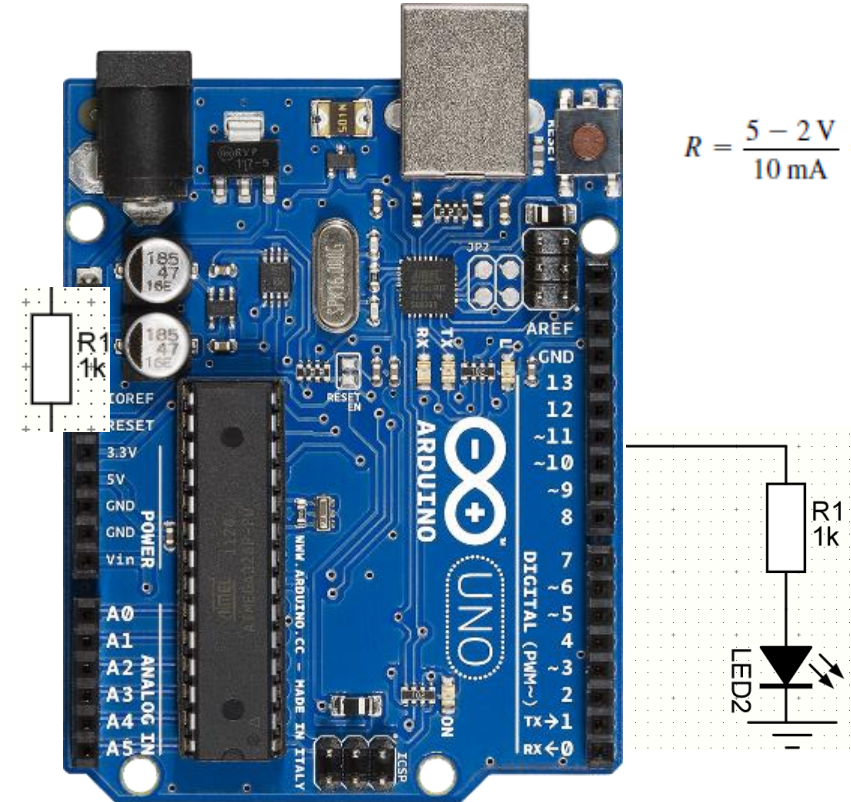


6 Entradas Analógicas (Pines 0 al 5)

Conexión de Cargas en Micros



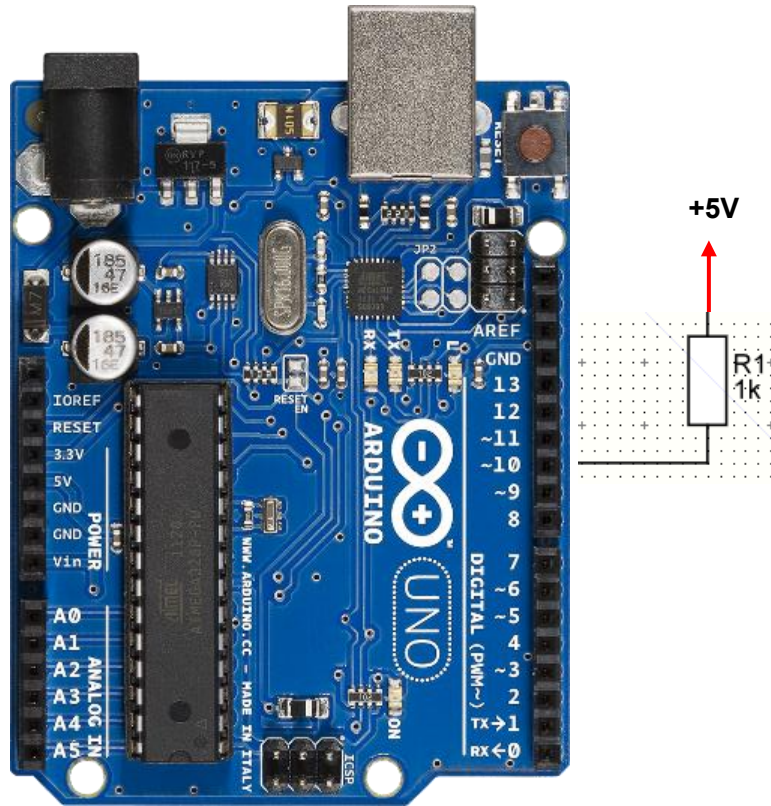
Current Sourcing Pull Down



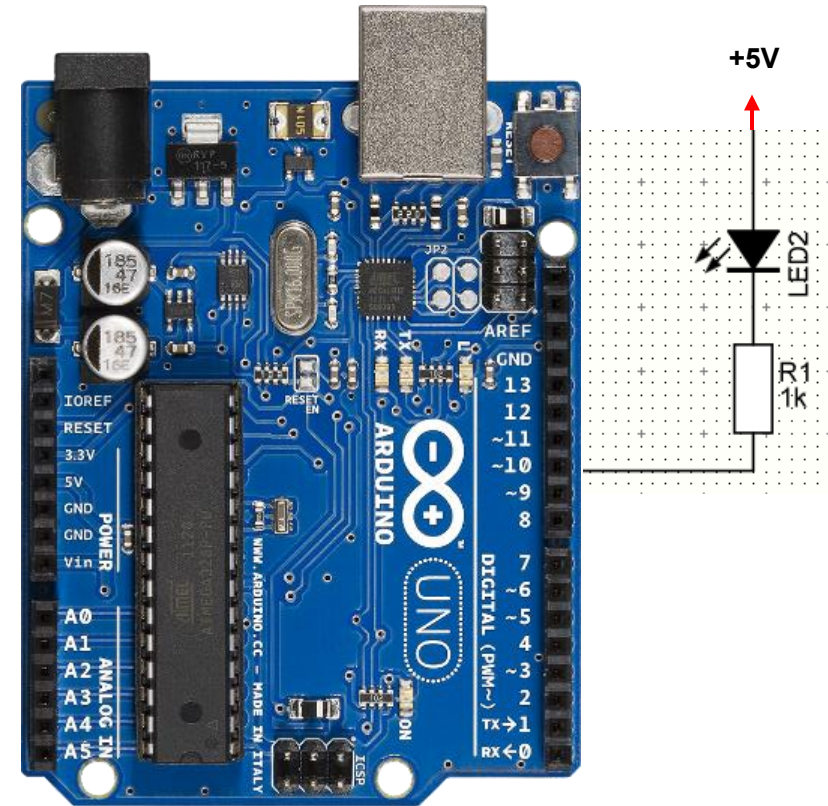
$$R = \frac{5 - 2 \text{ V}}{10 \text{ mA}} = \frac{3 \text{ V}}{10 \text{ mA}} = 0.3 \text{ K}$$

Conexión de LED en modo Source

Conexión de Cargas en Micros

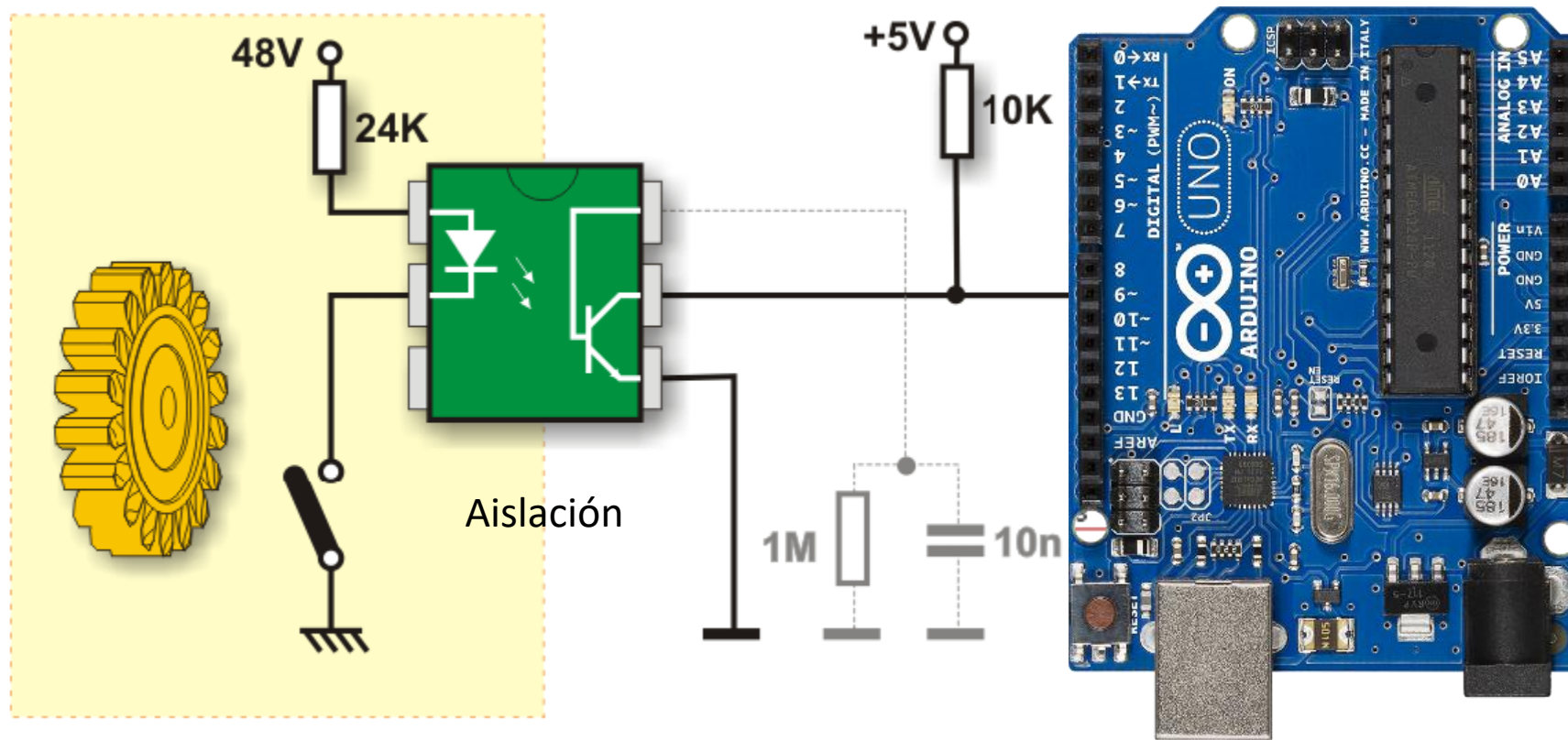


Current Sinking Pull Up

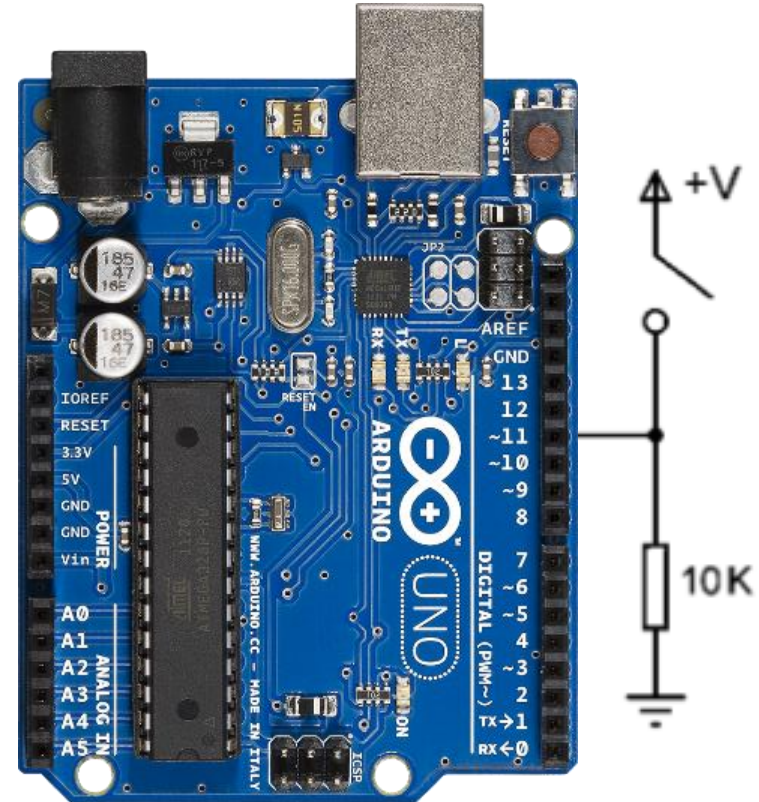
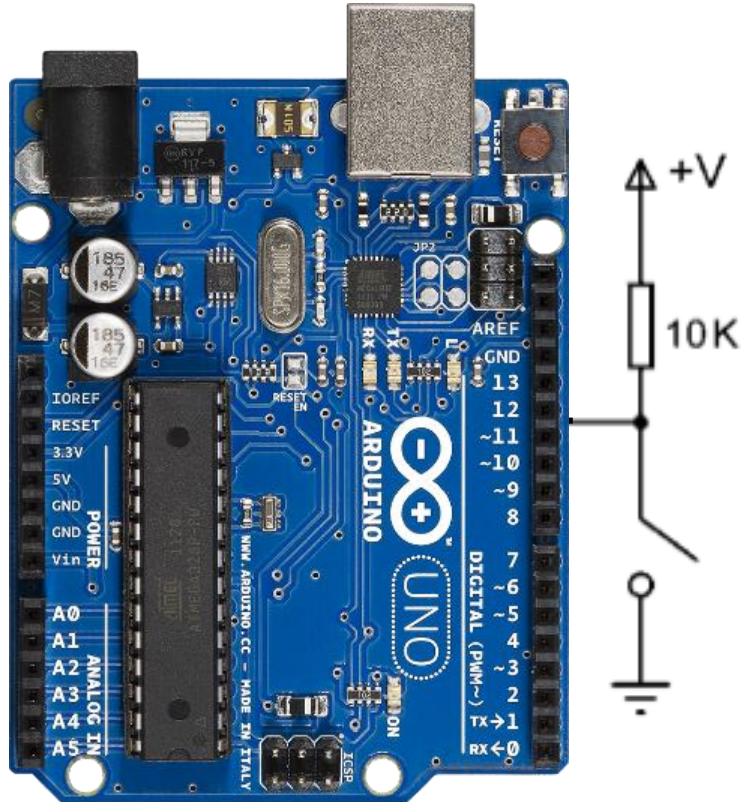


Conexión de LED en modo Sink

Aislación Óptica



Señal de Entrada

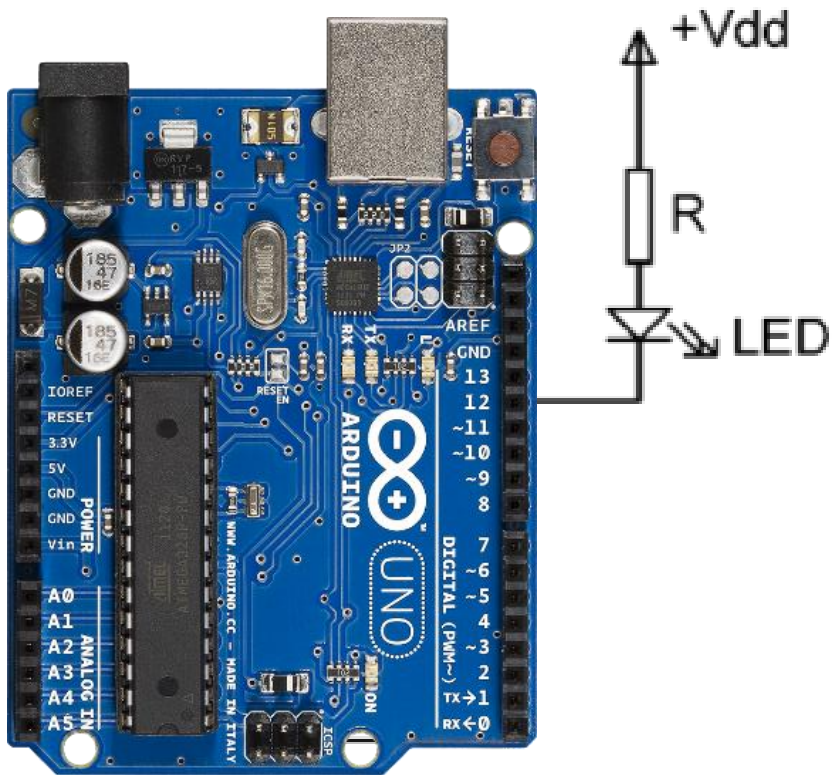


Señal de Salida con “Driver”

Tipicamente LED:

$I = 20 \text{ mA}$

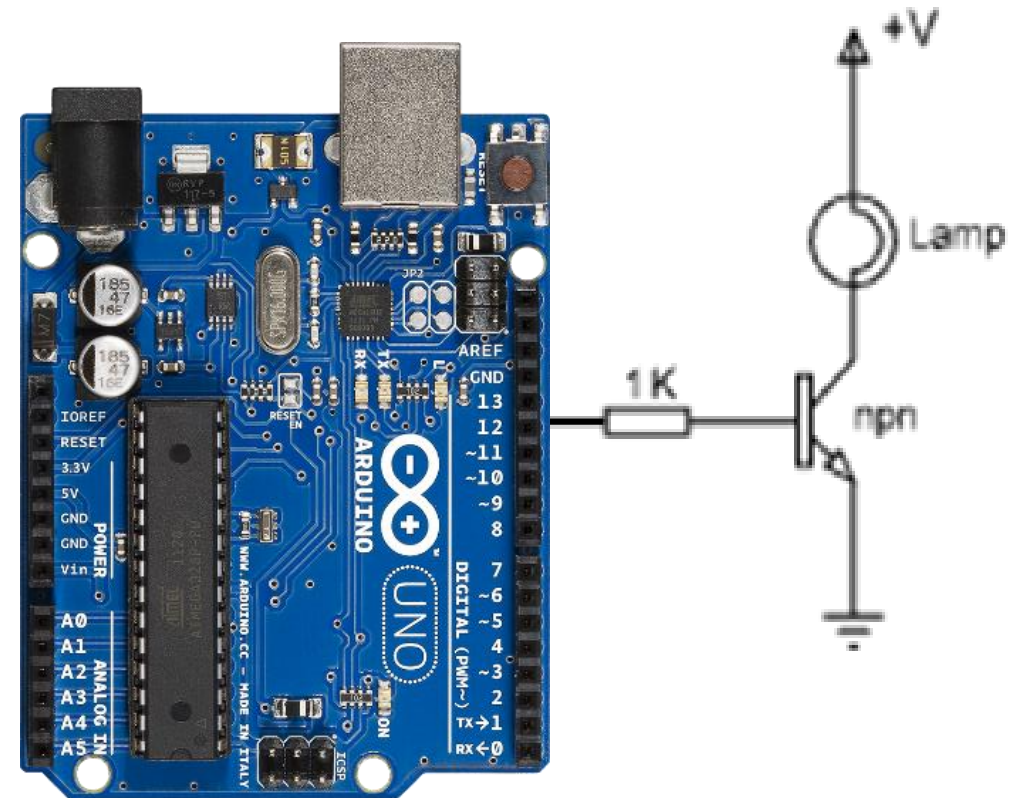
$V = 2 \text{ Volt}$



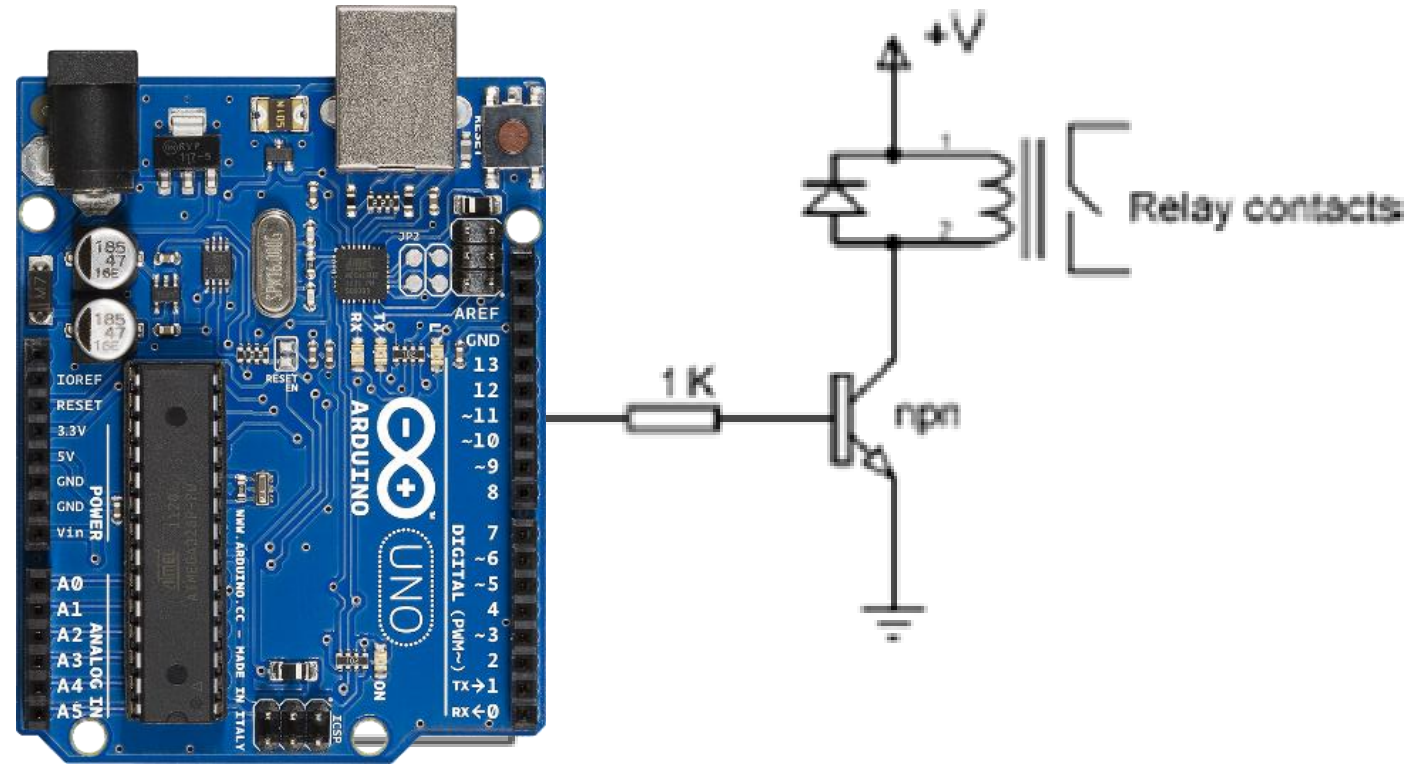
Uso de Lampara:

$I = 100 \text{ mA}$

$V = 9 \text{ Volt}$



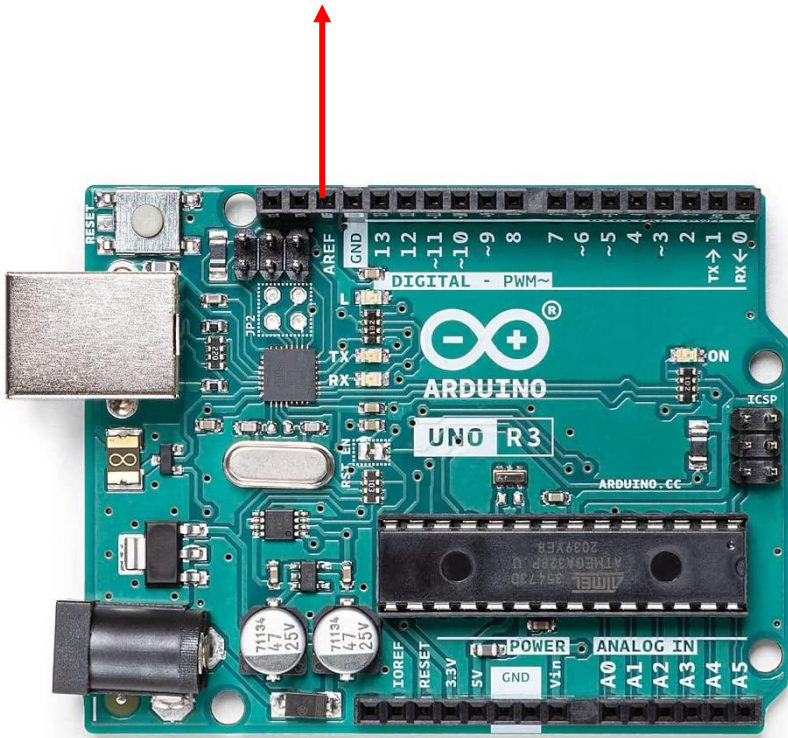
Señal de Salida con “Driver”



Arduino - Hardware

Señales Analógicas

AREF: Referencia del ADC



La resolución de un convertidor analógico digital (ADC) se expresa en número de bits.

$$\text{Resolución} = \frac{1}{2^n}$$

Por ejemplo, el Arduino tiene un convertidor analógico digital (ADC) de 10-bit tiene una resolución de

$$\text{Resolución} = \frac{1}{2^{10}} = \frac{1}{1024} = 0.00097$$

Si el rango de entrada es de 5V,

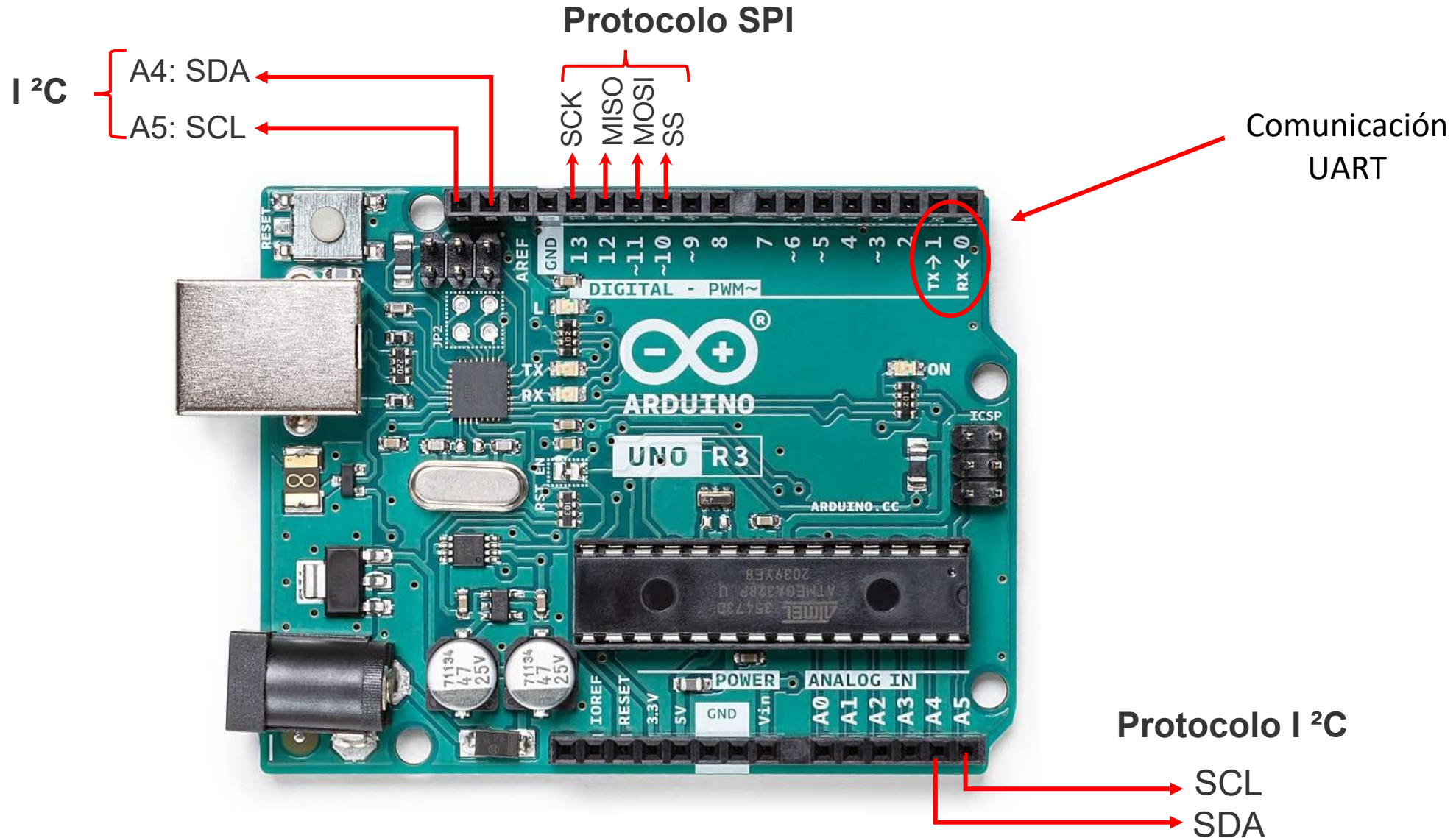
$$\frac{5V}{1024} = 0.00488 V = 4.88 mV$$

Lo que viene a decir que podremos medir variaciones de 4.88 mV en la señal analógica de entrada.

Si la señal es de 3.3 volts máxima tendremos que se variara de 0 a 614 cuentas. Con AREF la variación puede ser de 3.22mV

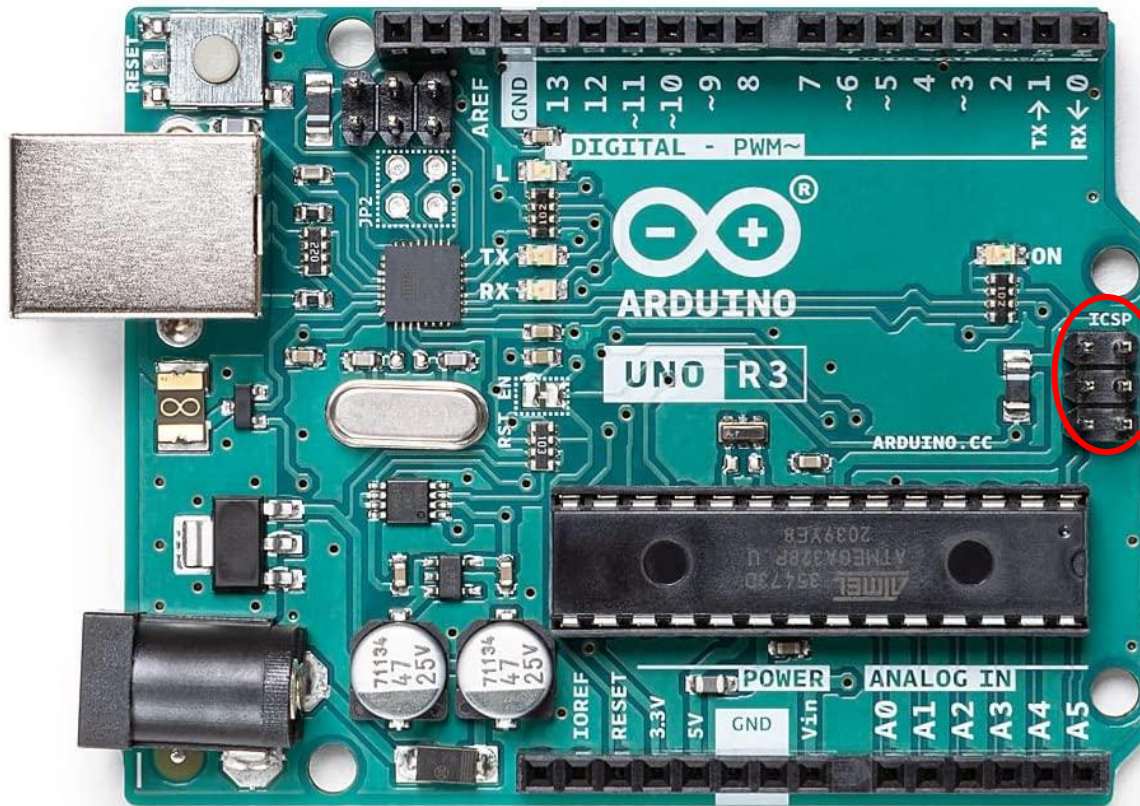
Arduino - Hardware

Señales Seriales



Arduino - Hardware

Programar
Bootloader



Puerto ICSP

Dando Vida a tus Ideas: El Entorno de Programación Arduino

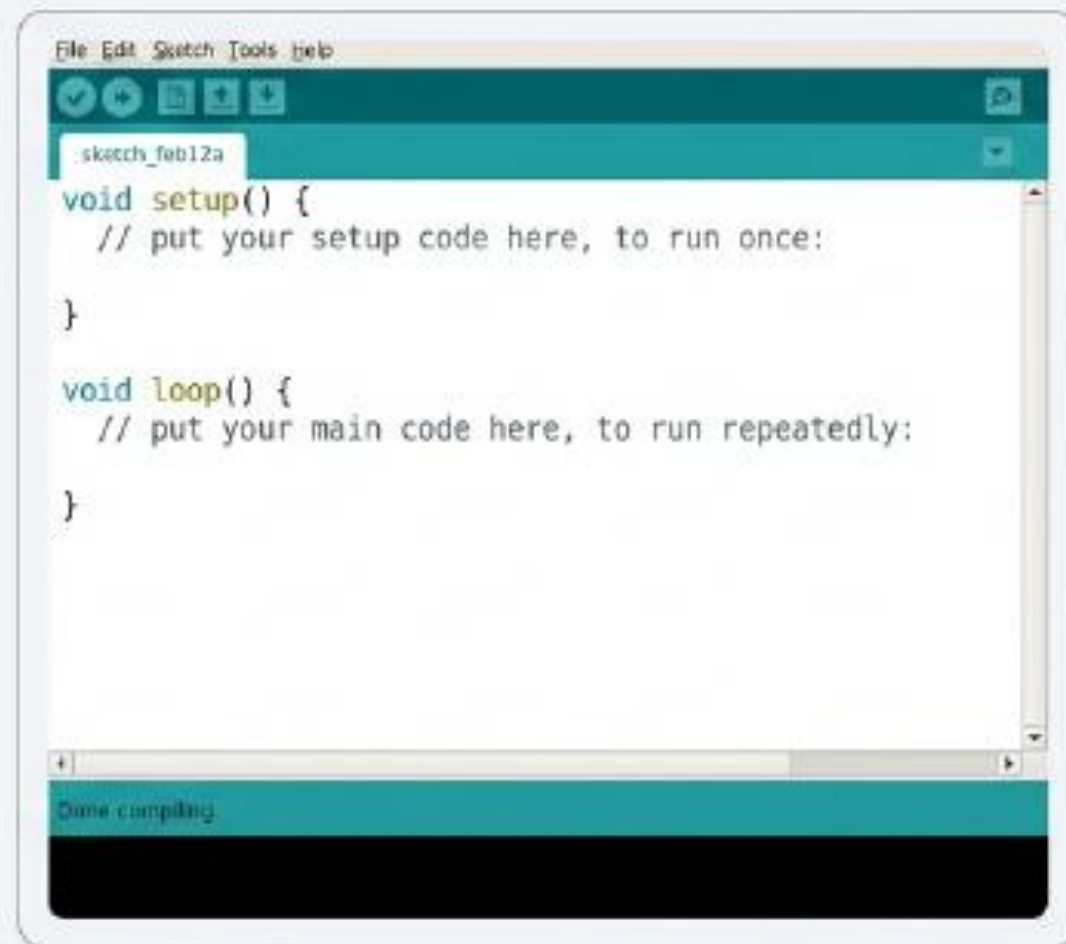
Lenguaje y IDE

Lenguaje: Basado en C++, pero simplificado a través del framework "Wiring". Se enfoca en la facilidad de uso.

```
// Se ejecuta una sola vez al inicio.  
void setup() {  
  // ... configuración inicial  
}  
  
// Se ejecuta continuamente, una y otra vez.  
void loop() {  
  // ... código principal  
}
```

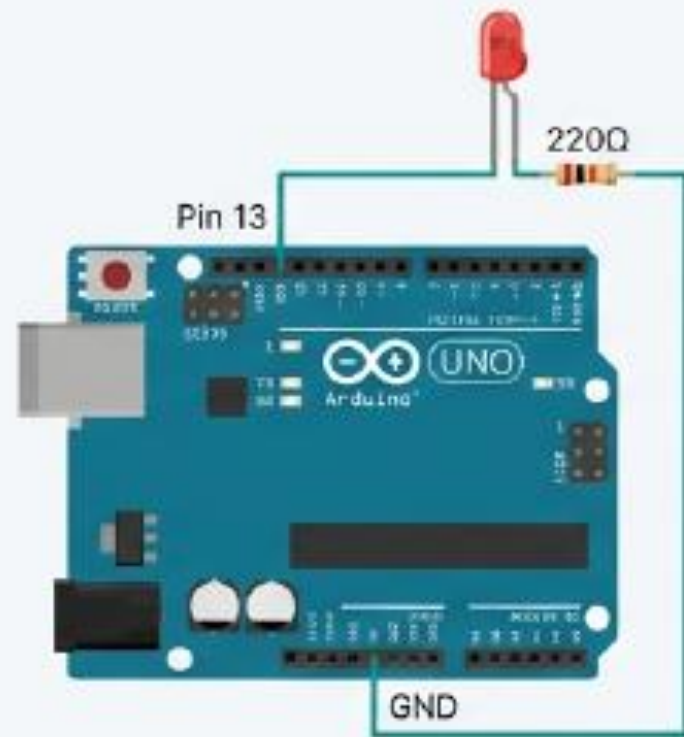
IDE: El Arduino IDE es un entorno de desarrollo integrado simple y multiplataforma para escribir, compilar y subir el código a la placa.

Captura del IDE



Tu Primer Programa: "Blink" Explicado

Diagrama del Circuito



Código Comentado

```
// La función setup() se ejecuta una vez al inicio del programa.  
void setup() {  
  // Configuramos el pin digital 13 como una SALIDA (OUTPUT).  
  pinMode(13, OUTPUT);  
}  
  
// La función loop() se ejecuta en un ciclo infinito después de setup().  
void loop() {  
  digitalWrite(13, HIGH); // Encendemos el LED (HIGH pone el pin en 5V).  
  delay(1000);             // Hacemos una pausa de 1000 milisegundos (1 segundo).  
  digitalWrite(13, LOW);  // Apagamos el LED (LOW pone el pin en 0V).  
  delay(1000);            // Hacemos otra pausa de 1 segundo.  
}
```


El Comunicador: ESP32

Texto Principal: Toma la fiabilidad de un microcontrolador y le añade el superpoder de la conectividad. Es el cerebro ideal para dispositivos que necesitan hablar con el mundo.

Puntos Clave:

- Wi-Fi y Bluetooth integrados.
- Potencia de procesamiento significativamente mayor.
- Eficiencia energética para proyectos a batería.



Anatomía del ESP32

Identidad:

El microcontrolador conectado. La puerta de entrada al IoT. Diseñado para ser el núcleo de proyectos de domótica, wearables y sensórica remota.

El Corazón (MCU):

Microcontrolador Tensilica Xtensa LX6 (Dual-Core). Permite ejecutar tareas de red en un núcleo y el programa principal en el otro.

Superpoderes:

- Conectividad Dual: Wi-Fi 802.11 b/g/n y Bluetooth (Clásico y BLE).
- Potencia: CPU de hasta 240 MHz.
- Bajo Consumo: Múltiples modos de 'deep sleep'.

El Lenguaje:

C++ (compatible con el entorno Arduino), MicroPython.
IDE: Arduino IDE, PlatformIO (con VScode), Thonny (para MicroPython).

La Familia



ESP32-WROOM-32: El módulo más popular



Variantes (S2, S3, C3): Optimizadas para diferentes necesidades (seguridad, bajo coste).

```
#include <WiFi.h>

const char* ssid = "TuRedWiFi";
const char* password = "TuContraseña";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Conectando a WiFi...");
  }
  Serial.println("Conectado!");
}

void loop() {
  // Tu código IoT aquí
}
```


El Poder del ESP32: Especificaciones Clave

El Corazón

Microcontrolador: Tensilica
Xtensa LX6 Dual-Core
Arquitectura: 32-bit



Especificaciones Vitales

Velocidad de Reloj: Hasta 240 MHz
SRAM: 520 KB
Modos de Bajo Consumo:
Múltiples 'sleep modes' para proyectos a batería.

Conectividad Integrada

Wi-Fi: 802.11 b/g/n
Bluetooth: v4.2 BR/EDR y BLE

Periféricos Avanzados

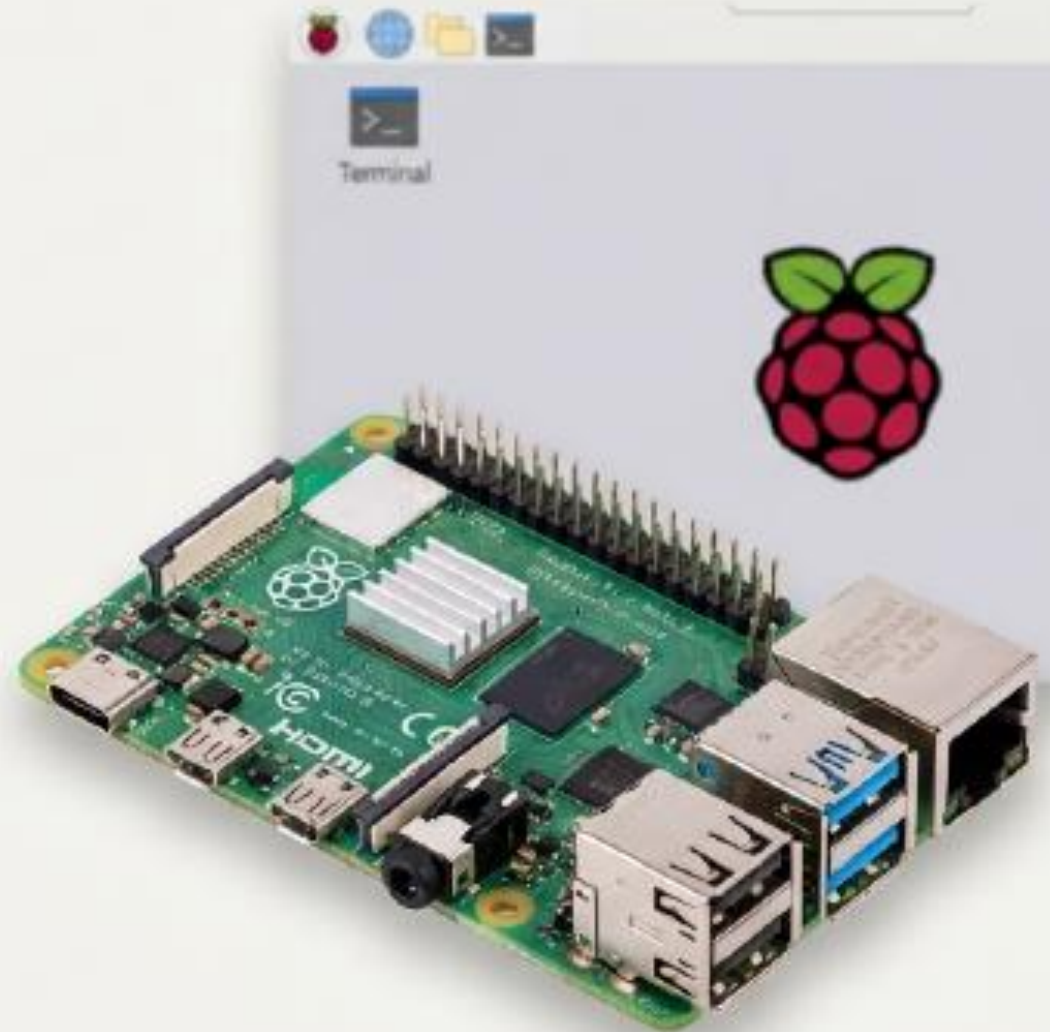
Múltiples pines ADC y DAC,
sensores táctiles capacitivos,
sensor de efecto Hall, y más.

El Razonador: Raspberry Pi

Texto Principal: Esto no es un microcontrolador, es un ordenador completo en miniatura. Capaz de ejecutar un sistema operativo, gestionar múltiples procesos y realizar tareas computacionalmente intensivas.

Puntos Clave:

- Ejecuta Linux (Raspberry Pi OS).
- Potencia para visión por computador, IA y servidores.
- Interfaz con periféricos estándar (HDMI, USB, Ethernet).



Anatomía de la Raspberry Pi

- **Identidad:** El ordenador de placa única (Single-Board Computer – SBC). Un cerebro para la orquestación de tareas complejas, no para el control determinista.
- **El Corazón (MPU):** Microprocesador Broadcom (ARM Cortex-A). Necesita componentes externos: RAM, almacenamiento (tarjeta SD), gestor de energía.
- **Superpoderes:**
 - Sistema Operativo:** Multitarea, gestión de archivos, red avanzada.
 - Procesamiento Gráfico:** Capaz de reproducir vídeo en 4K.
 - Comunidad Masiva:** Soporte y proyectos sin fin.
- **El Lenguaje:** Python es el rey. Pero también C++, Java, Node.js... cualquier lenguaje que corra en Linux.

La Familia



Pi 4 Model B: El buque insignia, máximo rendimiento.



Pi Zero 2 W: Pequeño, conectado e ideal para proyectos embebidos.



Pico: ¡Atención! Un microcontrolador (MCU), no un SBC. La incursión de Raspberry Pi en el mundo del control en tiempo real.

Anatomía de la Raspberry Pi 3 Model B

El Cerebro (MPU)

Procesador: Broadcom
BCM2837, Quad-Core ARM
Cortex-A53 64-bit @ 1.2GHz

Recursos de Computadora

RAM: 1GB LPDDR2
Almacenamiento: Ranura para tarjeta MicroSD



Conectividad y Multimedia

Red: Wi-Fi, Bluetooth 4.1/BLE,
Ethernet
Puertos: 4 x USB 2.0, Salida HDMI,
Audio/video compuesto, CSI, DSI.

Interfaz con el Mundo Físico

GPIO: Cabezal de 40 pines para conectar sensores, LEDs y otros componentes.

Un Universo de Software: Programando la Raspberry Pi



Sistema Operativo

La Raspberry Pi ejecuta un sistema operativo, principalmente **Raspberry Pi OS** (basado en Debian Linux). Esto significa que tienes un entorno de escritorio, navegador web, terminal y la capacidad de instalar software como en cualquier computadora.



Lenguajes de Programación

Python: Es el lenguaje oficial y preferido por la comunidad. Viene preinstalado y es ideal para scripting, robótica y ciencia de datos.

Se puede usar virtualmente cualquier lenguaje compatible con Linux: C/C++, Java, JavaScript (Node.js), etc.



Entornos de Desarrollo (IDEs)

Se puede programar desde un simple editor de texto como Sublime Text hasta IDEs más completos como VS Code, Thonny (para principiantes en Python) o **PyCharm**.

El Momento de la Verdad: Comparativa Directa

	Arduino UNO	ESP32	Raspberry Pi 3
 Categoría	Microcontrolador (MCU)	Microcontrolador (MCU)	Computadora de Placa Única (SBC)
 Cerebro	ATmega328P (8-bit @ 16MHz)	Xtensa LX6 (Dual-Core 32-bit @ 240MHz)	Broadcom BCM2837 (Quad-Core 64-bit @ 1.2GHz)
 Multitarea	No (Bucle único)	Limitada (RTOS)	Sí (Linux)
 Control en Tiempo Real	Excelente (Determinista)	Muy Bueno	Pobre (No determinista)
 Consumo Energético	Muy Bajo	Bajo (con modos sleep)	Alto
 Conectividad Integrada	No	Wi-Fi y Bluetooth	Wi-Fi, Bluetooth, Ethernet
 Periféricos de PC	No	No	Sí (HDMI, USB, Audio)
 Sistema Operativo	No	No (RTOS opcional)	Sí (Linux)
 Curva de Aprendizaje	Muy Fácil	Moderada	Avanzada
 Costo Estimado	\$	\$\$	\$\$\$

¿Qué Herramienta para qué Misión?



Tu Arsenal de Creación: Resumen Ejecutivo



Arduino

Elige **Arduino** para **controlar hardware** de forma simple y fiable. Es tu punto de partida para interactuar con el mundo físico.



ESP32

Elige **ESP32** cuando tu proyecto necesita **hablar con el mundo**. Es el cerebro ideal para el Internet de las Cosas.



Raspberry Pi

Elige **Raspberry Pi** cuando tu proyecto necesita la **inteligencia de una computadora**. Es la plataforma para procesamiento avanzado, multimedia y multitarea.

Instrucciones para instalación y familiarización con el IDE de Arduino

Arduino - IDE

Descargue el IDE de la siguiente Pagina web:

<https://www.arduino.cc/en/Main/Software>



ARDUINO 1.6.9

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software.

This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions.

Windows Installer

Windows ZIP file for non admin install

Mac OS X 10.7 Lion or newer

Linux 32 bits

Linux 64 bits

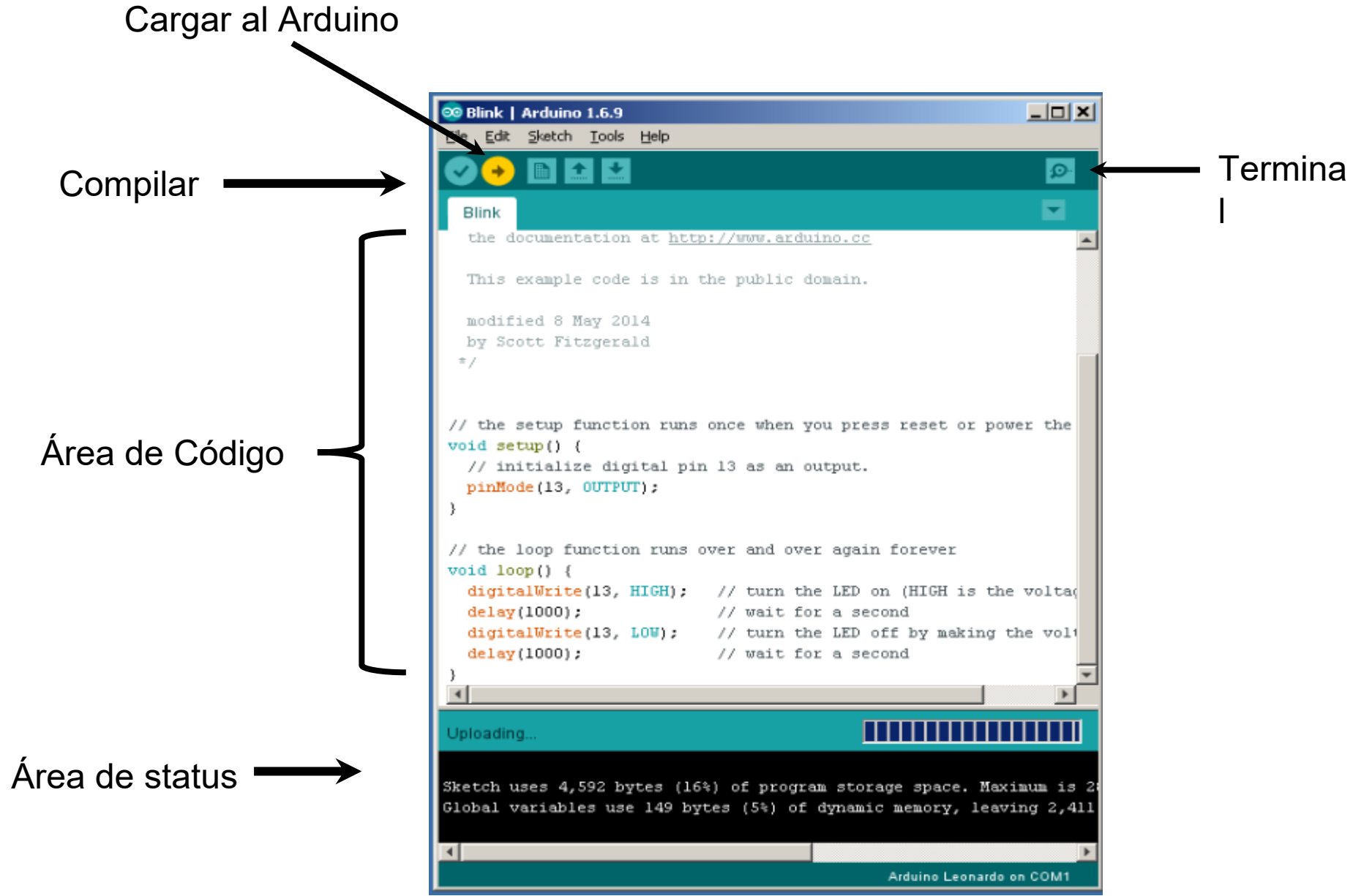
Linux ARM (experimental)

[Release Notes](#)

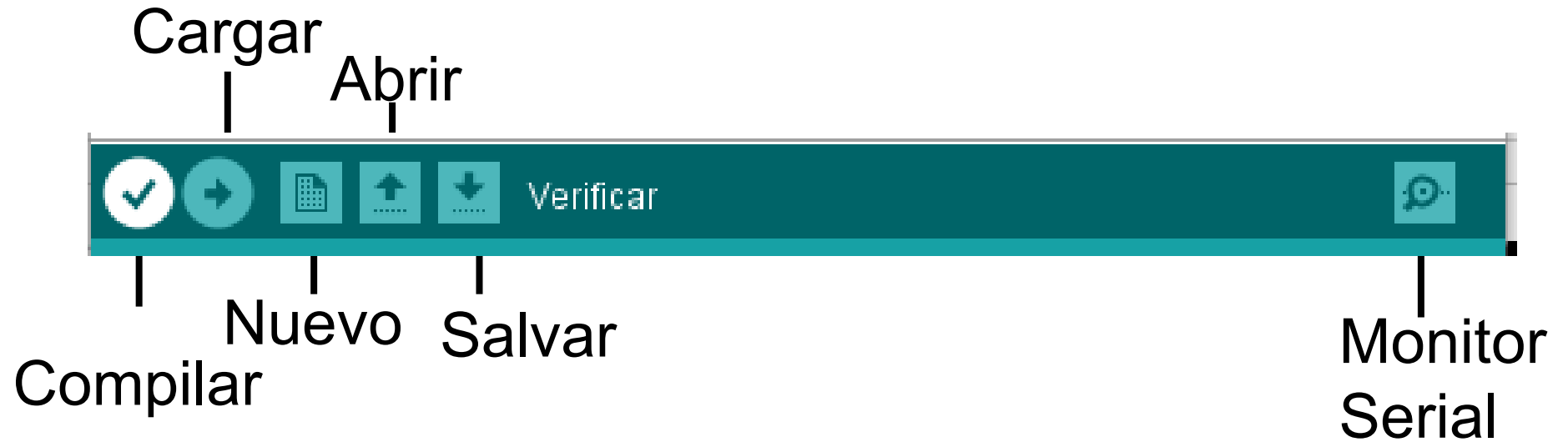
[Source Code](#)

[Checksums](#)

Ambiente IDE del Arduino



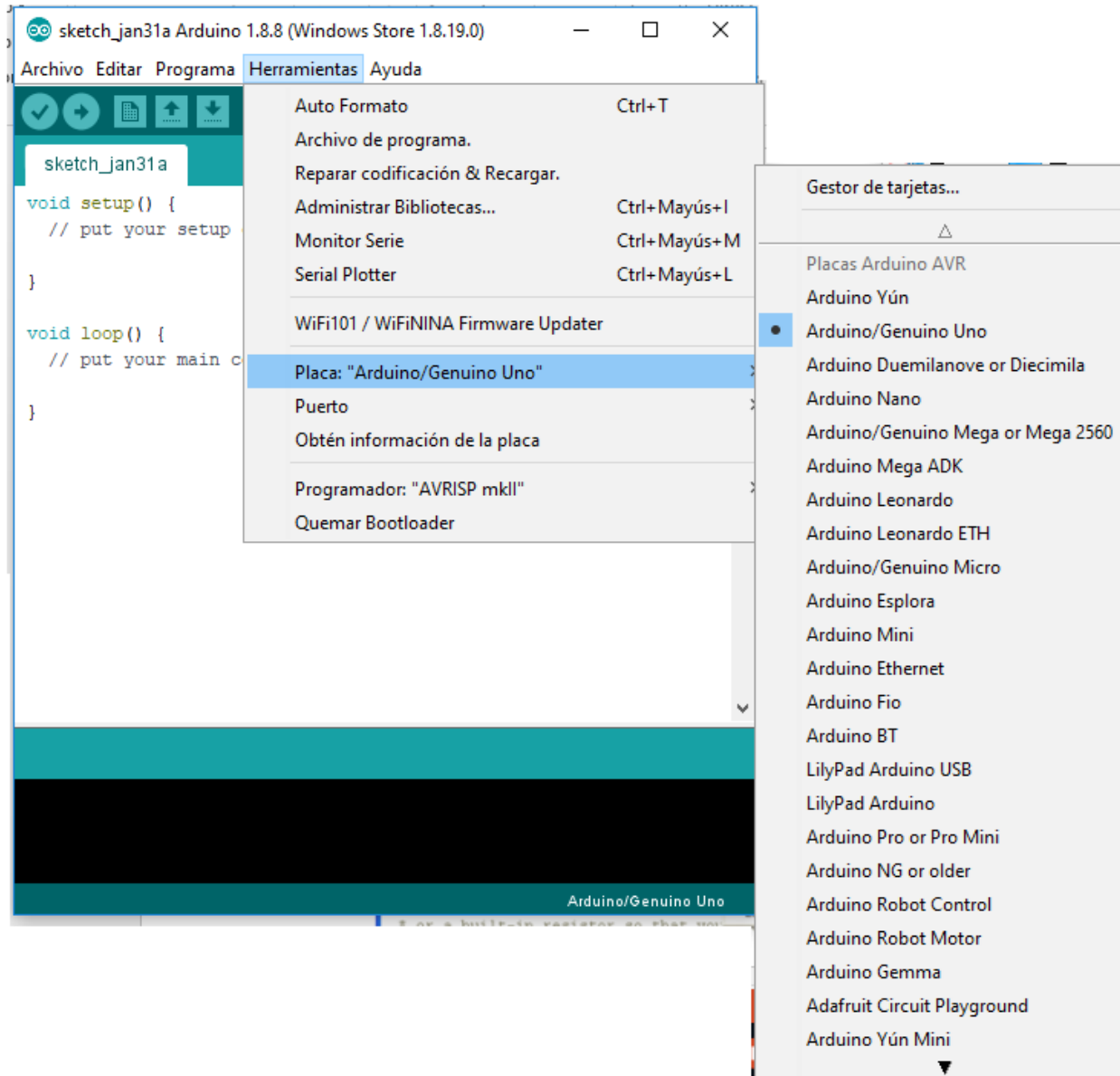
Ambiente IDE del Arduino



Ambiente IDE del Arduino

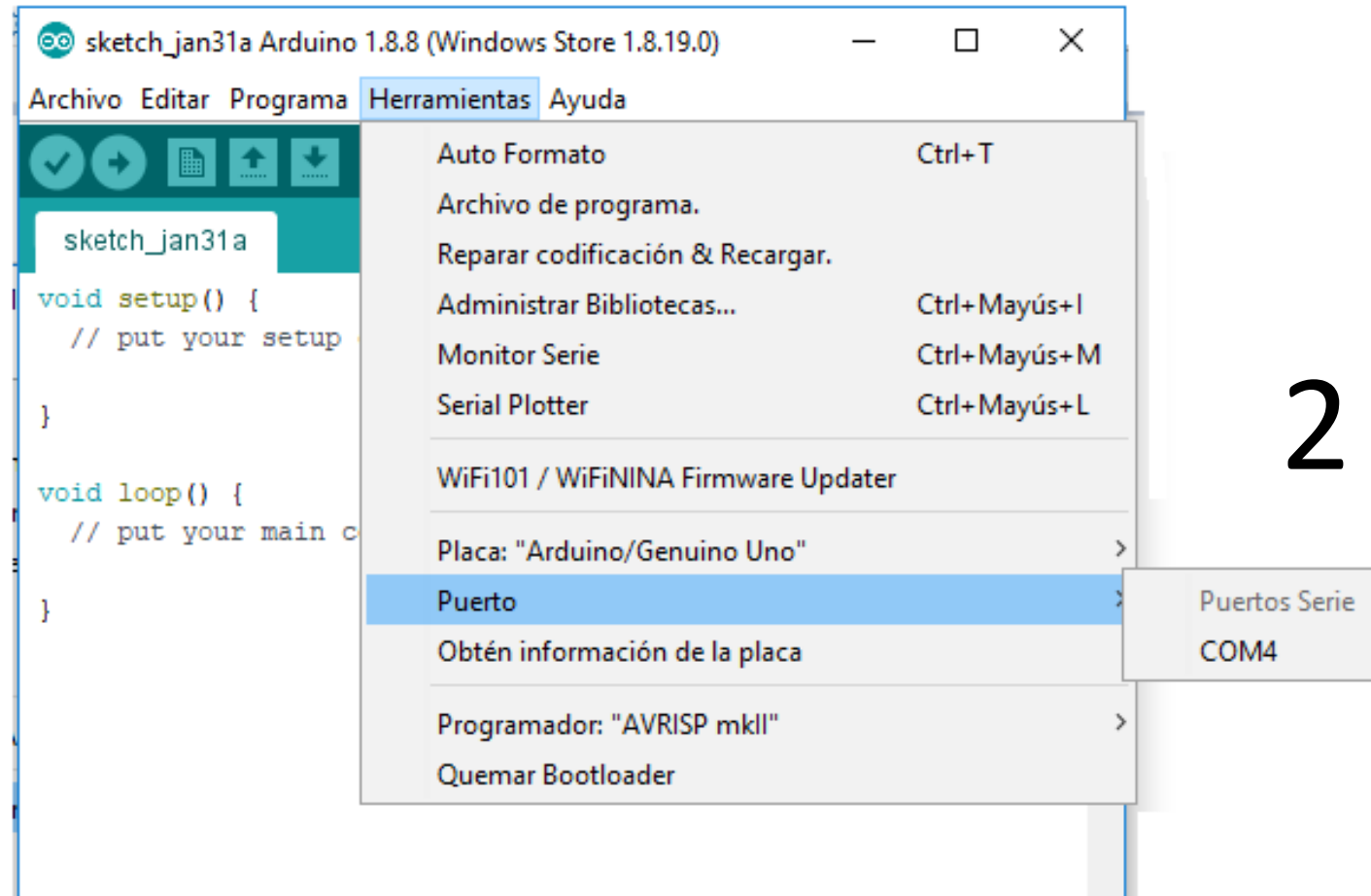
- >  Entradas y salidas de audio
- >  Equipo
- >  Intel(R) Dynamic Platform and Thermal Framework
- >  Monitores
- >  Mouse y otros dispositivos señaladores
- ▼  Otros dispositivos
 -  Dispositivo desconocido
- >  Procesadores
- ▼  Puertos (COM y LPT)
 -  Puerto de impresora ECP (LPT1)
 -  USB-SERIAL CH340 (COM4)
- >  Teclados
- >  Unidades de disco

Ambiente IDE del Arduino



1 Seleccione
el
Arduino

Ambiente IDE del Arduino



2 Seleccione
el Puerto

Ambiente IDE del Arduino

Escriba el Código →

```
void setup() {  
  pinMode(ledPin, OUTPUT); // sets t  
}  
void loop() {  
  digitalWrite(ledPin, HIGH); // sets t  
  delay(1000); // waits  
  digitalWrite(ledPin, LOW); // sets t  
  delay(1000); // waits  
}
```

Compile →



compile

Done compiling.

Cargue al Arduino →



upload

Tx/Rx led parpadearan →



TX/RX flash

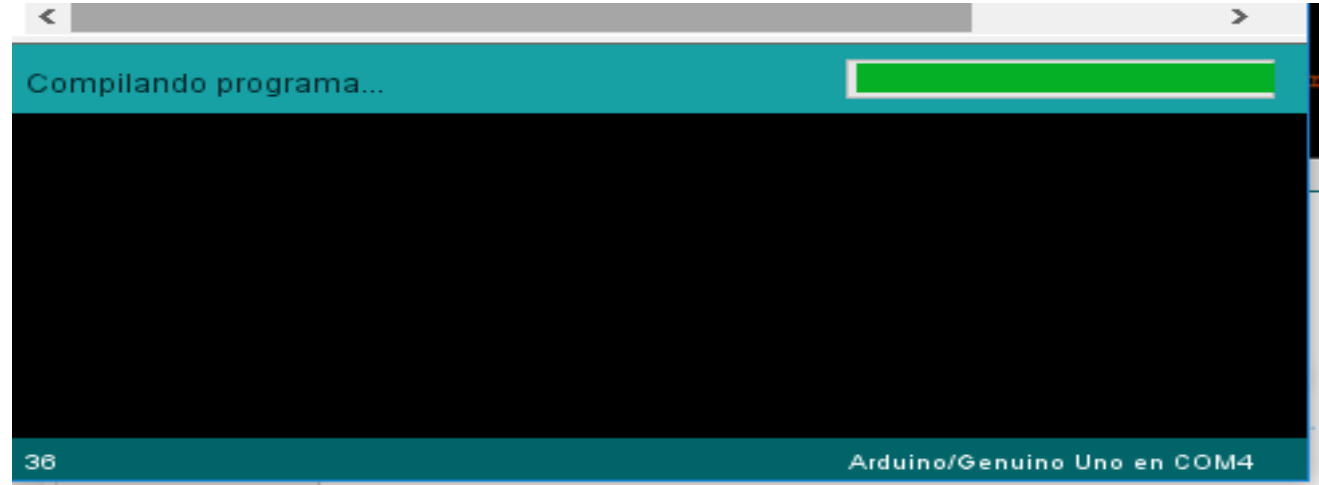
El Programa se inicia →



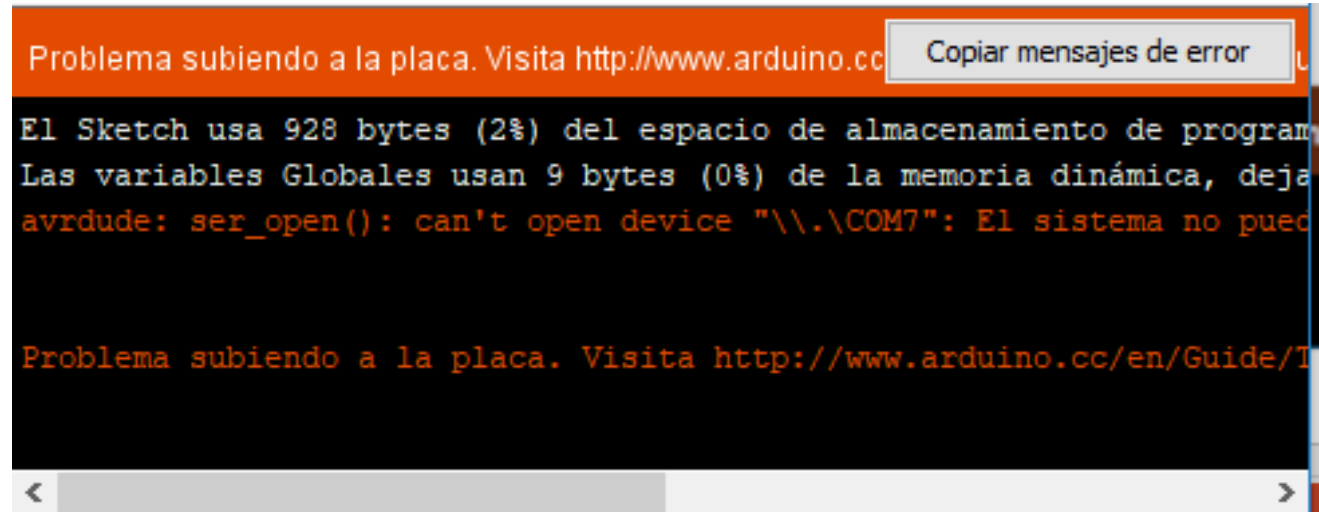
sketch runs

Mensajes de estado

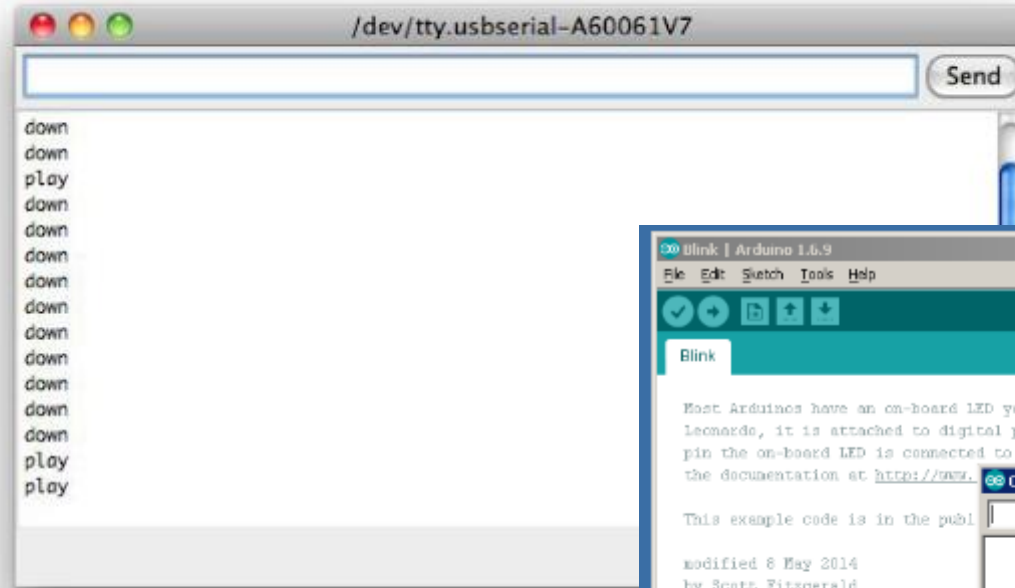
Carga en
Proceso



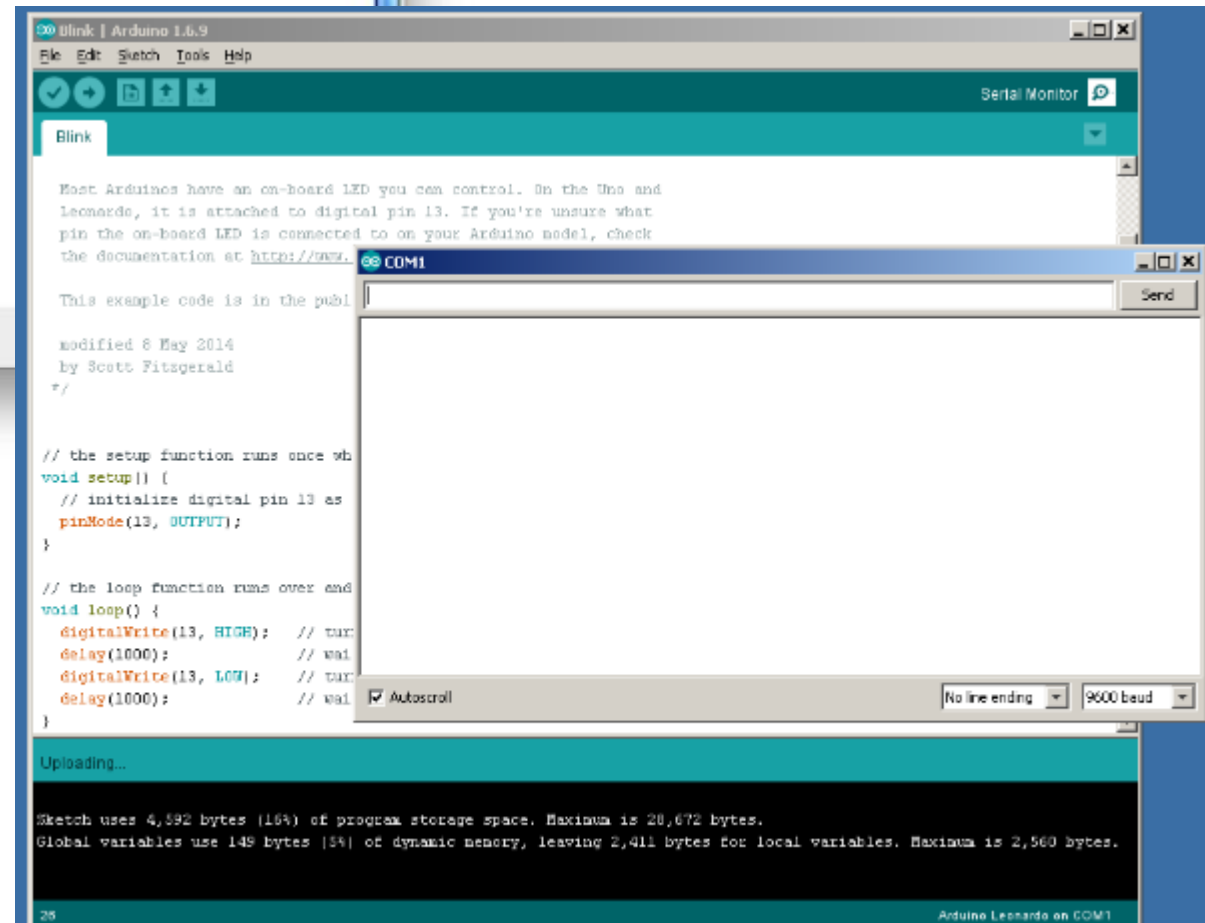
Puerto Serial
Equivocado



Ambiente IDE del Arduino



Termina



Arduino “Sketch”

Variables
Globales

❖ Declaración de Variables

Setup()

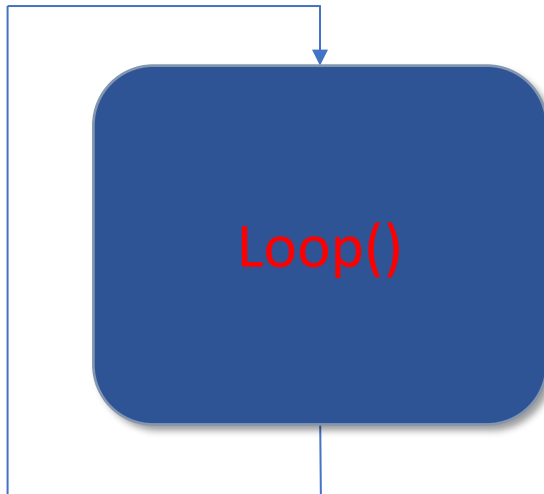
❖ Inicialización

❖ Setup() – Se ejecuta una sola vez al principio, definición de pines

Loop()

❖ Run

❖ Loop() – Se ejecuta repetidamente después del setup



Arduino “Sketch”

Variables Globales

```
/*  
 *      Nombre  
 *      Fecha  
 *      Notas  
 */
```

```
#include  
"DHT.h"  
#define DHTPIN 2
```

```
int LED1 = 13; // Asignado pin 13 a variable LED1
```

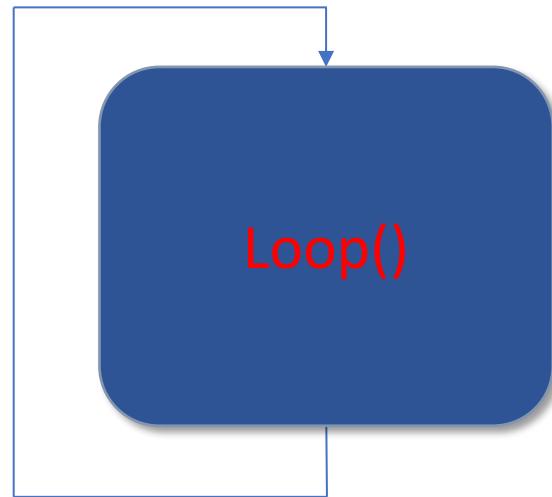
```
int Sensor1 = 0; // Asignado pin Analógico 0 a variable Sensor1
```

Arduino “Sketch”

Setup()

```
Void setup() {  
  pinMode(LED1, Output); // Asignado pin LED1 como salida  
  Serial.Begin(9600) // velocidad de comunicación con PC a 9600 b/s  
}
```

Arduino “Sketch”



```
Void loop() {
```

```
digitalWrite() – set a digital pin high/low
```

```
digitalRead() – read a digital pin's state
```

```
analogRead() – read an analog pin
```

```
analogWrite() – write an “analog” PWM value
```

```
delay() – wait an amount of time
```

```
millis() – get the current time
```

```
}
```

Codificación – Programación

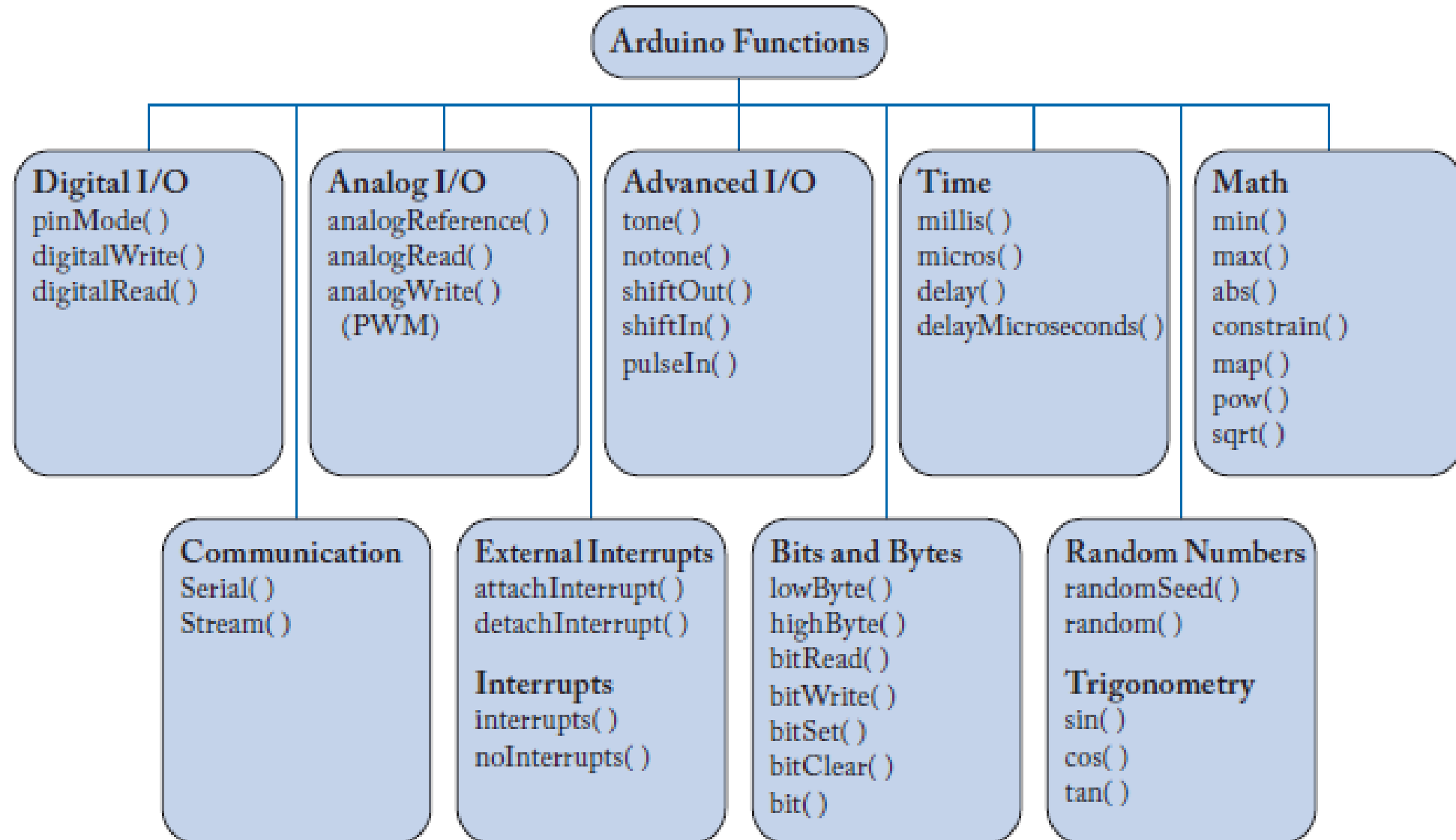
setup()

La función **setup()** es colocada al inicio del sketch. Se utiliza para inicializar Variables, modo de los Pines, utilizar librerías, etc. La función **setup ()** corre solo una vez, después de cada inicialización del Arduino o Reset.

loop()

Después de crear la función **setup()**, la función **loop()** Hace lo que su nombre indica que es realizar bucles.

Programación - Funciones



Programación – Tipo de Data

Numeric types	Bytes	Range	Use
<code>int</code>	2	–32768 to 32767	Represents positive and negative integer values.
<code>unsigned int</code>	2	0 to 65535	Represents only positive values; otherwise, similar to <code>int</code> .
<code>long</code>	4	–2147483648 to 2147483647	Represents a very large range of positive and negative values.
<code>unsigned long</code>	4	0 to 4294967295	Represents a very large range of positive values.
<code>float</code>	4	3.4028235E+38 to –3.4028235E+38	Represents numbers with fractions; use to approximate real-world measurements.
<code>double</code>	4	Same as <code>float</code>	In Arduino, <code>double</code> is just another name for <code>float</code> .
<code>boolean</code>	1	<code>false</code> (0) or <code>true</code> (1)	Represents true and false values.
<code>char</code>	1	–128 to 127	Represents a single character. Can also represent a signed value between –128 and 127.
<code>byte</code>	1	0 to 255	Similar to <code>char</code> , but for unsigned values.
Other types	Use		
<code>String</code>	Represents arrays of <code>chars</code> (characters) typically used to contain text.		
<code>void</code>	Used only in function declarations where no value is returned.		

Programación – Manejo de

caracteres

<code>charAt(n)</code>	Returns the <i>n</i> th character of the <code>String</code>
<code>compareTo(S2)</code>	Compares the <code>String</code> to the given <code>String</code> <i>S2</i>
<code>concat(S2)</code>	Returns a new <code>String</code> that is the combination of the <code>String</code> and <i>S2</i>
<code>endsWith(S2)</code>	Returns true if the <code>String</code> ends with the characters of <i>S2</i>
<code>equals(S2)</code>	Returns true if the <code>String</code> is an exact match for <i>S2</i> (case-sensitive)
<code>equalsIgnoreCase(S2)</code>	Same as <code>equals</code> but is not case-sensitive
<code>getBytes(buffer, len)</code>	Copies <code>len</code> characters into the supplied byte buffer
<code>indexOf(S)</code>	Returns the index of the supplied <code>String</code> (or character) or <code>-1</code> if not found
<code>lastIndexOf(S)</code>	Same as <code>indexOf</code> but starts from the end of the <code>String</code>
<code>length()</code>	Returns the number of characters in the <code>String</code>
<code>replace(A, B)</code>	Replaces all instances of <code>String</code> (or character) <i>A</i> with <i>B</i>
<code>setCharAt(index, c)</code>	Stores the character <i>c</i> in the <code>String</code> at the given index
<code>startsWith(S2)</code>	Returns true if the <code>String</code> starts with the characters of <i>S2</i>
<code>substring(index)</code>	Returns a <code>String</code> with the characters starting from <code>index</code> to the end of the <code>String</code>
<code>substring(index, to)</code>	Same as above, but the substring ends at the character location before the 'to' position
<code>toCharArray(buffer, len)</code>	Copies up to <code>len</code> characters of the <code>String</code> to the supplied buffer
<code>toInt()</code>	Returns the integer value of the numeric digits in the <code>String</code>
<code>toLowerCase()</code>	Returns a <code>String</code> with all characters converted to lowercase
<code>toUpperCase()</code>	Returns a <code>String</code> with all characters converted to uppercase
<code>trim()</code>	Returns a <code>String</code> with all leading and trailing whitespace removed

Comandos

Funciones Digitales

`pinMode(# pin, status)` : Definir el pin como Salida o entrada

`digitalWrite(# pin, HIGH/LOW)` : Colocar Salida digital high/low

`digitalRead(# Pin)` : Leer estado del pin read

`delay()` = Esperar

Ejemplo

```
int ledPin1 = 13;
```

‘ defino ledPin1 como puerto 13

```
Void setup() {
```

```
  pinMode (ledPin1, OUTPUT);
```

‘ Defino ledPin1 como puerto de salida

```
}
```

```
Void loop() {
```

```
  digitalWrite (ledPin1, HIGH);
```

‘ Coloco ledPin1 en “1”

```
  delay(500);
```

‘ Espero 500 milisegundos

```
  digitalWrite(ledPin1, LOW);
```

‘ Coloco “0” en LedPin1

```
  delay(500);
```

‘ Espero 500 mS

```
}
```


Comandos

Funciones Analogicas

`pinMode(# pin, status)` = Definir el pin como Salida o entrada

`analogRead(# pin)` = Leer Pin Analogico

`analogWrite(# pin, DutyCycle)` = Escribir sobre Pin Analogo

Ejemplo

```
int sensorPin = A0;
```

‘ defino sensorPin como puerto « 0 »

```
int ledPin = 13;
```

‘ defino ledPin como puerto 13

```
int sensorValue = 0;
```

```
Void setup() {
```

```
pinMode(ledPin, OUTPUT);
```

```
}
```

```
Void loop() {
```

```
sensorValue = analogRead(sensorPin);
```

```
digitalWrite(ledPin, HIGH);
```

```
delay(sensorValue);
```

```
digitalWrite(ledPin, LOW);
```

```
delay(sensorValue);
```

```
}
```

Comandos

Manejo del Tiempo

Delay(ms)

DelayMicroseconds(us)

millis() = Obtener el tiempo

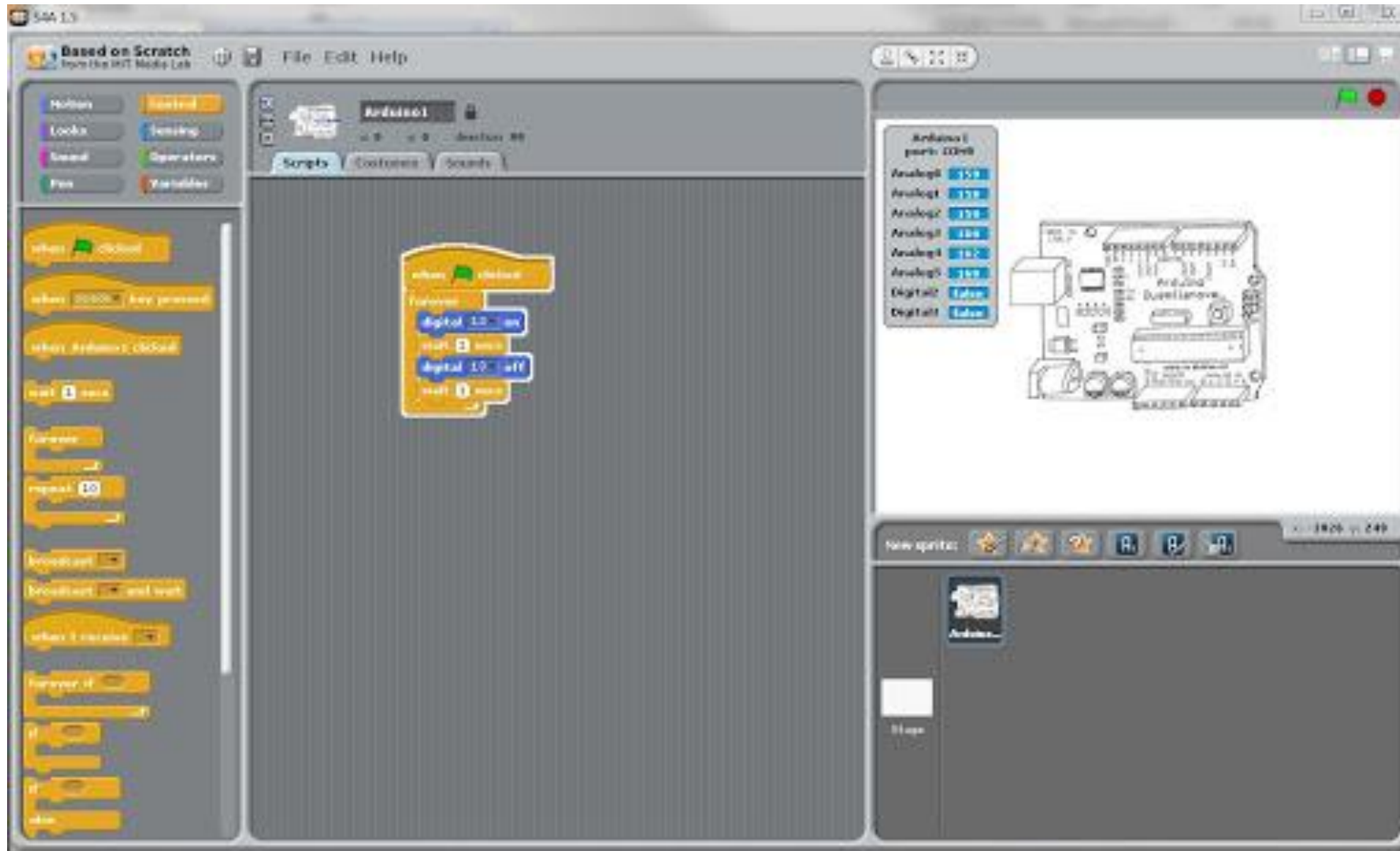
Proyecto #1 : Blink Led

```
int ledPin = 13;           // LED conectado al pin digital 13

void setup()               // Corre una vez, Cuando el sketch inicia
{
    pinMode(ledPin, OUTPUT); // coloca el pin Digital como salida
}

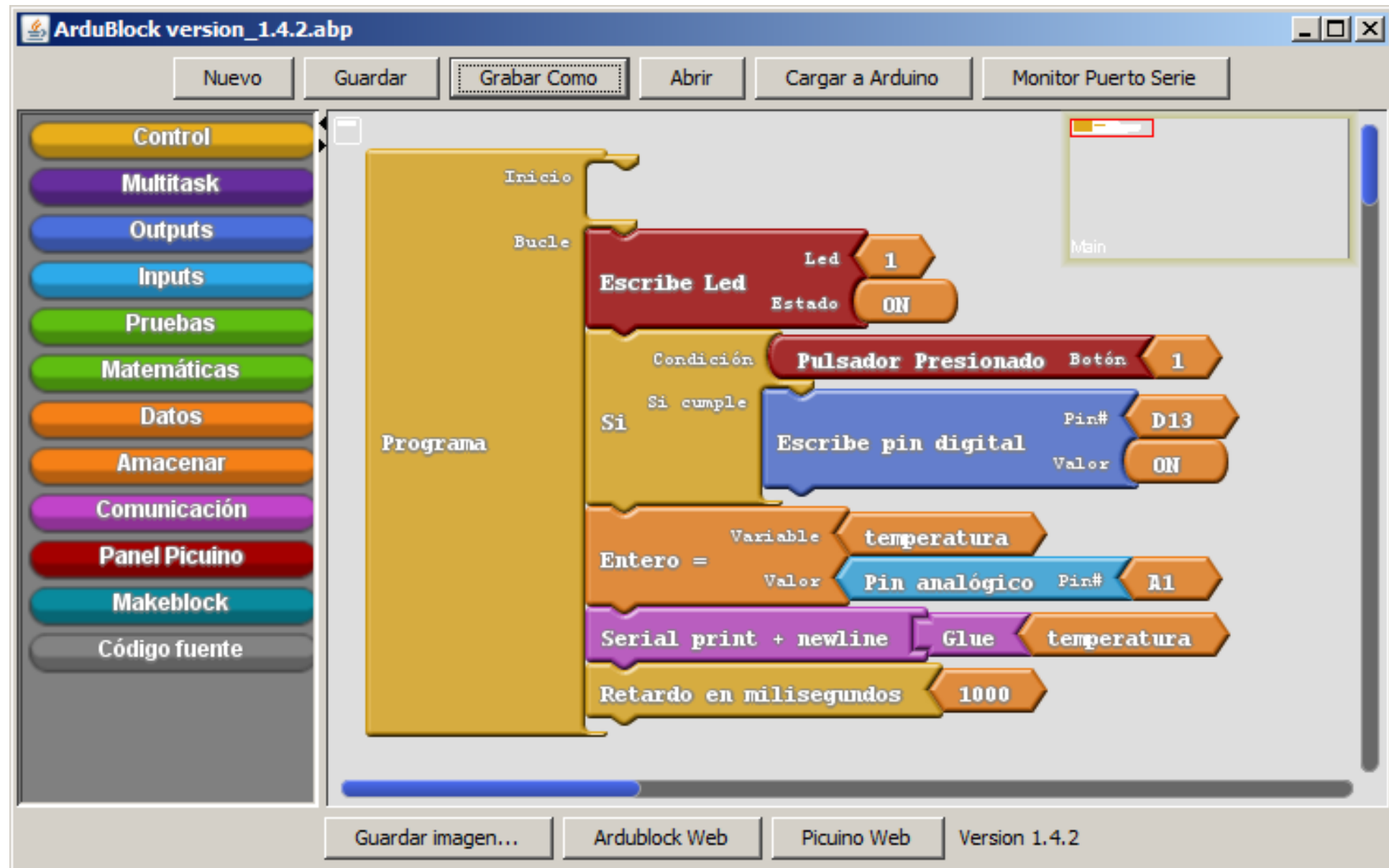
void loop()                // Corre en forma continua (bucle)
{
    digitalWrite(ledPin, HIGH); // Enciende el LED
    delay(1000);               // Espera 1 segundo
    digitalWrite(ledPin, LOW);  // Apaga el LED
}
```

Métodos de Programación Gráficos - Scratch



Métodos de Programación Gráficos - Ardublock

Ardublock



Simulador - WOKWI

WOKWI SAVE SHARE electronic-safe.ino by urish Docs SIGN IN

electronic-safe.ino README.md icons.h SafeState.h icons.cpp SafeState.cpp

diagram.json libraries.txt Library Manager

```
1 /**
2  * Arduino Electronic Safe
3  *
4  * Copyright (C) 2020, Uri Shaked.
5  * Released under the MIT License.
6  */
7
8 #include <LiquidCrystal.h>
9 #include <Keypad.h>
10 #include <Servo.h>
11 #include "SafeState.h"
12 #include "icons.h"
13
14 /* Locking mechanism definitions */
15 #define SERVO_PIN 6
16 #define SERVO_LOCK_POS 20
17 #define SERVO_UNLOCK_POS 90
18 Servo lockServo;
19
20 /* Display */
21 LiquidCrystal lcd(12, 11, 10, 9, 8, 7);
22
23 /* Keypad setup */
24 const byte KEYPAD_ROWS = 4;
25 const byte KEYPAD_COLS = 4;
26 byte rowPins[KEYPAD_ROWS] = {5, 4, 3, 2};
27 byte colPins[KEYPAD_COLS] = {A3, A2, A1, A0};
28 char keys[KEYPAD_ROWS][KEYPAD_COLS] = {
29   {'1', '2', '3', 'A'},
30   {'4', '5', '6', 'B'},
```

Simulation Description